PlacePro

Smart Placement Management System — Product Requirements Document

PlacePro Project Team

July 3, 2026

Contents

1 1. Product Vision

PlacePro aims to become the central, reliable desktop platform through which a college Training and Placement Office (TPO) manages the entire campus placement lifecycle — from student registration and company drive scheduling to application tracking, interview coordination, and final placement reporting.

Today, many institutions still rely on spreadsheets, email threads, and paper registers to coordinate placement activities.

This fragmented approach leads to data inconsistency, delayed communication, missed deadlines, and limited visibility for students and administrators alike. PlacePro addresses this gap by providing a structured,

database-backed desktop application that consolidates all placement operations into a single, role-aware interface.

The long-term vision is a system that:

- Eliminates manual record-keeping for placement coordinators
- Gives students transparent, real-time visibility into drives, applications, and outcomes
- Enables data-driven decision-making through centralized reports and analytics
- Serves as a demonstrable, production-quality academic project built on industry-standard Java desktop technologies

PlacePro is designed not merely as a software assignment, but as a practical tool that a placement cell could realistically deploy on a local network to streamline day-to-day operations during placement season.

---

2 2. Product Overview

One-liner: PlacePro is a desktop-based placement management system that enables colleges to register students, publish company drives, track applications, schedule interviews, and generate placement reports — all from a unified Java Swing interface backed by a MySQL database.

Field	Detail
Product Name	PlacePro
Tagline	Smart Placement Management System
Application Type	Desktop (Java Swing)
Database	MySQL (via JDBC)
Target Institution	Engineering and degree colleges with a dedicated placement cell
Primary Users	Students, Placement Officers, Administrators, Recruiters
Deployment Model	Local installation on college lab/office machines
Version Control	Git / GitHub

2.1 2.1 Description

PlacePro is a full-featured desktop application developed for managing campus placement activities end to end.

The system provides role-based access for students, placement officers, administrators, and recruiters. Placement officers and administrators can maintain student records, onboard recruiting companies, create and publish placement drives, manage application workflows, and record interview outcomes. Students can view eligible drives, submit applications, track status updates, and access their placement history. Recruiters can view assigned shortlists, coordinate interviews, and record outcomes through a restricted portal.

The application is built using Java with Java Swing for the graphical user interface, JDBC for database connectivity, and MySQL as the persistent data store. Source code is managed through Git and hosted on GitHub, supporting collaborative development and version tracking throughout the academic project lifecycle.

PlacePro is intended for use during active placement seasons, academic exhibitions, and project evaluations — demonstrating practical software engineering skills while solving a real operational problem faced by college placement offices.

---

3 3. One-line Description

PlacePro is a Java Swing desktop application that centralizes campus placement operations — student profiles, company drives, applications, interviews, and reports — into one MySQL-backed system for colleges and placement coordinators.

---

4 4. Product Goals

1. **Centralize Placement Data** — Consolidate student records, company information, drive schedules, and application statuses into a single MySQL database accessible through one desktop application.
2. **Automate Manual Workflows** — Replace spreadsheet-based tracking with structured forms, validation, and automated status updates for applications and interview rounds.
3. **Enable Role-Based Access** — Provide distinct interfaces and permissions for students, placement officers, recruiters, and administrators, ensuring data integrity and appropriate visibility.
4. **Improve Transparency for Students** — Allow students to view published drives, check eligibility, submit applications, and monitor status without depending on manual email or notice-board updates.
5. **Support Reporting and Auditability** — Generate placement summaries, drive-wise statistics, and student outcome reports that administrators can use for internal review and accreditation documentation.
6. **Demonstrate Industry-Standard Development Practices** — Deliver a well-structured Java desktop application using JDBC, layered architecture, and Git-based version control suitable for academic evaluation and technical demonstration.

---

5 5. Business Goals

1. **Reduce Administrative Overhead** — Minimize the time placement officers spend on manual data entry, duplicate record maintenance, and status reconciliation across multiple tools.
2. **Increase Placement Process Efficiency** — Accelerate drive creation, student shortlisting, and interview scheduling by providing a single source of truth for all stakeholders.
3. **Improve Data Accuracy** — Eliminate inconsistencies caused by disconnected spreadsheets and ad-hoc communication channels during high-volume placement seasons.
4. **Enhance Student Experience** — Provide students with timely, self-service access to placement opportunities and application progress, reducing uncertainty and inquiry load on the placement cell.
5. **Enable Institutional Reporting** — Equip the placement office with exportable reports and summaries required for management reviews, department heads, and external audit requirements.
6. **Establish a Reusable Academic Asset** — Create a maintainable codebase and documentation set that future batches can extend, forming a lasting contribution to the institution's technical portfolio.

---

6 6. Success Criteria

#	Criterion	Measurement
SC-01	**Functional completeness**	All core modules (authentication, student management, company/drive management, applications, reporting) are implemented and demonstrable end-to-end.
SC-02	**Data persistence**	All records are stored and retrieved correctly from MySQL via JDBC with no data loss across application restarts.

#	Criterion	Measurement
SC-03	**Role-based access**	Student, officer, recruiter, and administrator roles each enforce correct permissions; unauthorized actions are blocked at the application layer.
SC-04	**Usability**	A new user (student or admin) can complete their primary task (apply to a drive / create a drive) within 5 minutes without external documentation.
SC-05	**Performance**	Standard CRUD operations and report generation complete within 3 seconds on a typical college lab machine with up to 500 student records.
SC-06	**Reliability**	Application handles invalid input, database connection failures, and duplicate submissions gracefully with user-friendly error messages.
SC-07	**Code quality**	Source code is organized in a layered structure (UI, service/DAO, database), version-controlled on GitHub, and includes meaningful commit history.
SC-08	**Academic deliverables**	PRD, working application, database schema, and demo readiness are completed within the project timeline and evaluation requirements.

---

7 7. Scope of the Project

7.1 7.1 In Scope

Area	Included
Platform	Desktop application built with Java and Java Swing
Database	MySQL database with JDBC connectivity
User roles	Student, Placement Officer, Administrator, Recruiter
Notifications	In-app notifications (F14); external email/SMS excluded (see Section 7.2)
Student management	Registration, profile maintenance, eligibility criteria
Company management	Company profiles, job roles, package details
Drive management	Create, publish, update, and close placement drives
Applications	Student apply, officer/admin review, shortlist/reject workflow
Interview tracking	Schedule rounds, record outcomes (selected/rejected/on hold)
Authentication	Login, session management, role-based navigation
Reporting	Placement summaries, drive-wise and department-wise statistics
Version control	Git repository hosted on GitHub

7.2 7.2 Out of Scope

Area	Excluded	Rationale
Web or mobile clients	Browser-based or Android/iOS apps	Project scope is limited to Java Swing desktop
Cloud deployment	AWS, Azure, or SaaS hosting	Local desktop + local/network MySQL only
Online payment processing	Registration fees, company payments	Not required for core placement workflow
Video interview integration	Zoom, Teams, or WebRTC	External tooling; not part of desktop scope
Resume parsing / AI matching	Automated skill-to-job matching	Advanced feature; deferred to future versions

Area	Excluded	Rationale
Multi-college federation	Cross-institution placement portals	Single-institution scope for v1.0
Email/SMS notifications	External email or SMS delivery	In-app notifications (F14) are in scope; only automated external delivery is deferred to Section 19
OAuth / social login	Google, LinkedIn authentication	Local credential-based auth only

7.3 7.3 Assumptions

- MySQL server is installed and accessible on the deployment machine or local network.
- Placement administrators are responsible for data entry accuracy for companies and drives.
- Students receive login credentials from the placement cell upon registration.
- Recruiter accounts are created by administrators or placement officers and linked to a registered company.
- The application runs on machines with Java Runtime Environment (JRE) 11 or higher installed.

---

8 8. Problem Statement

Core Problem: College placement offices lack a unified, reliable system to manage the full placement lifecycle. Student records, company drives, applications, and interview outcomes are scattered across spreadsheets, email chains, and physical registers — creating delays, data errors, and poor visibility for students, placement officers, and recruiting companies alike.

Despite the critical importance of campus placements to institutional reputation and student outcomes, most colleges continue to coordinate this high-stakes process using manual, disconnected tools that were never designed for multi-stakeholder workflow management at scale.

8.1 8.1 Current Problems by Stakeholder

Stakeholder	Key Problems
Students	No single source of truth for available drives, eligibility criteria, or application deadlines. Status updates depend on notice boards, class representatives, or informal WhatsApp groups. Students miss opportunities due to delayed or inconsistent communication. Difficulty tracking application history across multiple companies and interview rounds.
Placement Officers	Heavy reliance on Excel sheets and paper forms for student data, company details, and shortlists. Duplicate data entry across multiple files when drives are created, students apply, and results are recorded. No real-time dashboard to monitor drive participation, shortlist progress, or placement statistics. High risk of data loss, version conflicts, and transcription errors during peak placement season.
Recruiters	Receive student shortlists in inconsistent formats (Excel, PDF, email attachments) with varying fields and quality. Limited visibility into how many eligible students exist before visiting campus. Coordination of interview schedules requires back-and-forth communication with the placement cell. Delayed feedback loops when recording selection outcomes, affecting offer letter processing and follow-up drives.

8.2 8.2 The Existing Manual Process

The typical placement workflow at institutions without dedicated software follows a repetitive, paper- and spreadsheet-driven cycle:

1. **Student data collection** — Placement officers collect student profiles (academic records, contact details, resumes) via Google Forms or physical forms at the start of the academic year. Data is consolidated into a master Excel workbook.
2. **Company onboarding** — Recruiting companies contact the TPO via email or phone. Company details, job descriptions, eligibility criteria, and visit dates are recorded manually in separate spreadsheets or Word documents.
3. **Drive announcement** — Officers filter eligible students from the master sheet based on CGPA, branch, and backlogs. Notices are posted on bulletin boards, shared in department WhatsApp groups, or sent via bulk email. Students respond by submitting hard-copy or email applications.
4. **Application and shortlisting** — Officers maintain a drive-specific Excel file listing applicants. Shortlists are prepared manually, cross-referenced against eligibility rules, and shared with recruiters via email — often in non-standard formats.
5. **Interview coordination** — Interview schedules are prepared in spreadsheets and communicated verbally or through class representatives. Room assignments and panel details are managed separately.
6. **Result recording** — Selection outcomes are collected from recruiters (verbally, via email, or on paper), manually entered into result sheets, and used to update the master placement record — often days after the actual interview.
7. **Reporting** — End-of-season placement statistics are compiled by manually aggregating data from multiple files, a process that is time-consuming and prone to calculation errors.

This process repeats for every company drive throughout the placement season, multiplying administrative effort and error risk with each cycle.

8.3 8.3 Key Inefficiencies

Inefficiency	Description	Impact
Fragmented data storage	Student, company, and application data exist in separate files with no relational integrity.	Duplicate records, orphaned entries, and conflicting versions of the same data.
Manual eligibility checks	Officers manually filter students against CGPA, branch, and backlog criteria for each drive.	Hours of repetitive work per drive; human error leads to ineligible students being shortlisted or eligible students being missed.
Delayed status communication	Application and interview status updates require manual outreach to individual students or class groups.	Students operate with incomplete information; placement cell is flooded with repetitive status inquiries.
No audit trail	Changes to spreadsheets are not tracked; there is no history of who modified a record or when.	Disputes over shortlist decisions and placement records are difficult to resolve.
Report generation overhead	Placement summaries require manual copy-paste and formula work across multiple workbooks at season end.	Reports are delayed, inaccurate, or incomplete when leadership needs timely insights.
Single-point dependency	Critical placement data often resides on one officer's laptop or a shared drive with inconsistent backups.	Hardware failure or accidental deletion can cause irreversible data loss mid-season.
Recruiter coordination friction	Shortlists and schedules are exchanged through unstructured email threads.	Miscommunication, format mismatches, and scheduling conflicts during on-campus drives.

8.4 8.4 Why Current Methods Are Not Effective

Existing approaches — whether spreadsheets, generic project management tools, or ad-hoc communication channels — fail to meet the specific demands of campus placement management for the following reasons:

1. **Not purpose-built for placement workflows** — Spreadsheets are flexible but lack enforced data models, role-based access, validation rules, and workflow states (applied → shortlisted → interviewed → selected). Officers must invent conventions for each drive, leading to inconsistency.
2. **No real-time synchronization** — When one officer updates a shortlist in a local Excel file, other stakeholders do not see the change until the file is manually shared. Students and recruiters operate on stale information.
3. **Poor scalability during peak season** — A college running 30–50 company drives per season with hundreds of students generates thousands of application records. Manual tools degrade rapidly under this volume; lookup, filtering, and reporting become impractical.
4. **Inadequate access control** — Shared spreadsheets and email distribution cannot enforce that students see only their own applications, or that only authorized officers modify drive configurations. Sensitive student data is exposed through broad file sharing.
5. **No integration between entities** — Students, companies, drives, applications, and interview results are logically related but stored in disconnected artifacts. There is no foreign-key integrity, no cascade updates, and no single queryable dataset.
6. **Enterprise tools are overkill or inaccessible** — Commercial HR and applicant tracking systems require cloud subscriptions, complex configuration, and ongoing licensing costs — impractical for a college desktop environment and a student development project.
7. **Student experience does not meet expectations** — Students accustomed to self-service digital portals in other domains (admissions, examinations) expect the same for placements. Manual processes create frustration and reduce trust in the placement cell.

In summary, the gap is not merely a lack of software — it is the absence of a **structured, role-aware, database-backed desktop system** tailored to the placement domain and deployable within a college's existing infrastructure.

---

9 9. Proposed Solution

PlacePro addresses the problems identified above through a dedicated desktop application that centralizes placement operations into a single MySQL-backed system with role-based Java Swing interfaces for students, placement officers, administrators, and recruiters.

9.1 9.1 Solution Architecture

PlacePro is organized into three logical layers:

Layer 1 — Data Persistence: A normalized MySQL database stores all entities — students, companies, drives, applications, interview schedules, and placement outcomes — with relational integrity enforced through primary and foreign keys. JDBC provides type-safe connectivity between the application and the database.

Layer 2 — Business Logic and Data Access: A service layer encapsulates validation rules, eligibility checks, application state transitions, and report generation. A JDBC-based DAO layer handles all MySQL persistence on behalf of the service layer (detailed layer breakdown in Section 17). Placement officers and administrators interact with management modules; students interact with a restricted student portal — all backed by the same consistent data model.

Layer 3 — Presentation: A Java Swing desktop interface delivers form-based data entry, searchable tables, status dashboards, and exportable reports. The UI is designed for placement cell workstations and student lab machines on a local network, requiring no internet dependency for core operations.

9.2 9.2 How PlacePro Resolves Each Problem Area

Problem Area	PlacePro Resolution
Fragmented records	Single MySQL database as the authoritative source for all placement data.
Manual eligibility filtering	Drive configuration stores eligibility rules; system filters eligible students automatically.
Delayed student updates	Students log in to view drive listings, submit applications, and track status in real time.
Shortlist management	Officers review applications within the app, update statuses, and generate standardized shortlists for recruiters.
Interview coordination	Interview rounds and outcomes are recorded against applications within the system.
End-of-season reporting	Built-in reports aggregate placement statistics by company, department, and drive without manual spreadsheet work.
Access control	Role-based login ensures each user role sees only the data and actions appropriate to their permissions.
Data safety	Persistent storage in MySQL with standard backup practices replaces fragile single-file spreadsheet dependencies.

9.3 9.3 Why This Approach Works

- **Purpose-built** — Designed specifically for campus placement workflows, not adapted from generic tools.
- **Low deployment barrier** — Runs as a desktop application on existing college lab machines with a local or network MySQL instance; no cloud subscription required.
- **Familiar technology stack** — Java, Swing, JDBC, and MySQL are industry-standard, well-documented, and appropriate for academic project evaluation.
- **Immediate operational value** — Officers gain a working admin console from day one; students gain self-service access to drives and application tracking.
- **Extensible foundation** — Layered architecture and version-controlled codebase (Git/GitHub) allow future enhancements such as notifications, web access, or resume uploads without redesigning the core data model.

PlacePro transforms placement management from a fragmented, manual exercise into a structured, auditable, and efficient process — benefiting students, placement officers, and recruiters through a single integrated desktop platform.

---

10 10. Target Users

PlacePro serves four distinct user groups involved in the campus placement ecosystem. Each group has unique responsibilities, objectives, and pain points that the system is designed to address. The table below summarizes persona types; detailed profiles follow in subsections 10.1–10.4.

Persona	Role Type	Primary Interaction with PlacePro
Student	Primary End User	Logs in to view drives, apply, and track application status
Placement Officer	Primary End User	Manages drives, reviews applications, coordinates interviews, records outcomes
Recruiter	External Stakeholder	Logs in via Recruiter Login (F5) to view shortlists, schedules, and record interview outcomes for assigned companies

Persona	Role Type	Primary Interaction with PlacePro
Administrator	System Operator	Manages user accounts, system configuration, data integrity, and oversight

---

10.1 10.1 Student

Attribute	Detail
Role	Registered student eligible for campus placement drives (typically final-year undergraduate or postgraduate)
Persona Name	*Priya Sharma — Final-year CSE student*

Responsibilities

- Maintain an accurate academic and contact profile as required by the placement cell
- Monitor published placement drives and eligibility criteria
- Submit applications to drives for which they qualify
- Attend scheduled interviews and assessment rounds as per drive timelines
- Report selection outcomes and offer details to the placement cell when required

Goals

- Discover all relevant placement opportunities in one place without relying on informal channels
- Understand eligibility requirements before investing time in an application
- Submit applications quickly and receive timely confirmation
- Track application status (applied, shortlisted, interviewed, selected, rejected) without repeated visits to the placement office
- Build a clear record of placement activity for personal reference and graduation requirements

Pain Points

- Drive announcements scattered across notice boards, emails, and messaging groups — easy to miss deadlines
- Unclear eligibility rules leading to wasted effort on ineligible applications
- No self-service visibility into application or interview status; must ask officers or class representatives
- Anxiety and uncertainty during peak season due to delayed or inconsistent updates
- Difficulty maintaining a personal history of applications across dozens of company drives

How PlacePro Helps

- Provides a student login with a dashboard listing all active drives filtered by eligibility
- Displays drive details — company, role, package, criteria, and deadlines — in a structured format
- Enables one-click application submission with instant confirmation stored in the database
- Shows real-time application status updates as officers progress records through the workflow
- Maintains a personal application history accessible at any time from the desktop client

---

10.2 10.2 Placement Officer

Attribute	Detail
Role	Training and Placement Officer (TPO) or placement cell staff responsible for day-to-day placement operations
Persona Name	*Mr. Rajesh Kumar — Head, Training & Placement Cell*

Responsibilities

- Register and maintain student records with academic eligibility data
- Onboard recruiting companies and capture job role, package, and visit details
- Create, publish, and manage placement drives with defined eligibility criteria
- Review incoming student applications and prepare shortlists for recruiters
- Coordinate interview schedules, venues, and panel assignments during on-campus drives
- Record interview outcomes and update final placement records
- Generate placement statistics and reports for institutional leadership

Goals

- Run the entire placement season efficiently with minimal manual data handling
- Ensure accurate eligibility enforcement and fair, auditable shortlist decisions
- Reduce time spent answering repetitive student status inquiries
- Deliver standardized, professional shortlists and schedules to recruiting companies
- Produce accurate end-of-season reports without last-minute spreadsheet consolidation
- Maintain data integrity and backup throughout the high-pressure placement period

Pain Points

- Overwhelming volume of manual data entry and cross-referencing across multiple Excel files
- Error-prone eligibility filtering when hundreds of students apply to each drive
- Constant interruptions from students asking for application status updates
- Format inconsistencies when sharing shortlists with different recruiters
- Risk of data loss or version conflicts when multiple files are edited concurrently
- Days of effort required to compile placement reports at season end

How PlacePro Helps

- Centralizes student, company, drive, and application data in a single MySQL database
- Automates eligibility filtering based on rules defined at drive creation
- Provides an admin console to review applications, update statuses, and manage workflows in one interface
- Generates exportable shortlists in a consistent format for recruiter handoff
- Records interview rounds and outcomes directly against application records
- Produces on-demand placement reports by company, department, and drive — eliminating manual aggregation

---

10.3 10.3 Recruiter

Attribute	Detail
Role	Human resources representative or hiring manager from a company conducting an on-campus or pooled campus placement drive
Persona Name	*Ms. Ananya Desai — Talent Acquisition Lead, TechCorp Solutions*

Responsibilities

- Coordinate with the college placement cell to schedule campus visit dates and drive logistics
- Provide job descriptions, eligibility criteria, compensation details, and selection process requirements
- Review student shortlists provided by the placement cell before the visit
- Conduct assessment rounds, technical interviews, and HR interviews on campus or virtually
- Communicate selection results and offer details back to the placement officer
- Ensure a smooth candidate experience that reflects positively on the employer brand

Goals

- Receive an accurate, complete shortlist of eligible candidates before arriving on campus
- Minimize pre-visit coordination overhead with the placement cell
- Access consistent student data (academic records, contact details, branch) in a standard format
- Run interviews on a clear schedule without logistical confusion
- Provide timely selection feedback so offer processing can begin without delay
- Evaluate the college as an efficient, professional placement partner for future drives

Pain Points

- Shortlists received in varying Excel formats with missing or inconsistent fields
- Difficulty assessing the size of the eligible talent pool before committing to a visit
- Back-and-forth email exchanges to finalize schedules, venues, and candidate lists
- Delayed or incomplete information when recording who was selected versus rejected
- Perception of disorganization when the placement cell cannot quickly answer candidate count or status questions

How PlacePro Helps

- Enables placement officers to generate standardized, complete shortlists derived from verified database records
- Ensures eligibility has been system-validated before candidates appear on a shortlist, reducing unqualified applicants
- Allows officers to share accurate drive schedules and candidate counts promptly during coordination
- Supports structured recording of interview outcomes so recruiters' feedback is captured immediately in the system
- Provides recruiters with a dedicated login portal (F5) to view shortlisted candidates, confirm interview schedules, and record round-wise outcomes for assigned drives
- Improves overall coordination quality, making the college a more reliable placement partner

---

10.4 10.4 Administrator

Attribute	Detail
Role	System administrator or senior placement cell authority responsible for platform governance, user management, and institutional oversight
Persona Name	*Dr. Sunita Menon — Dean of Student Affairs / IT Liaison*

Responsibilities

- Provision and deactivate user accounts for students, placement officers, and recruiters
- Configure system-level settings (database connection, academic year, department list)
- Oversee data integrity, backup procedures, and access control policies
- Monitor system usage and ensure compliance with institutional data handling guidelines

- Review aggregated placement reports for accuracy before submission to management or accreditation bodies
- Coordinate with IT staff for MySQL server maintenance, deployment, and troubleshooting

Goals

- Ensure the placement system is secure, stable, and available throughout the placement season
- Maintain clear separation of roles so students cannot access admin functions and vice versa
- Protect sensitive student personal and academic data from unauthorized access or loss
- Have confidence in report accuracy when presenting placement outcomes to institutional leadership
- Support a maintainable system that future academic batches can operate and extend

Pain Points

- No centralized control when placement data lives in personal spreadsheets on individual laptops
- Inability to audit who changed a record or when a shortlist was modified
- Security risks from broadly shared files containing student personal information
- Dependence on a single officer's machine with no formal backup or recovery process
- Difficulty verifying report accuracy when underlying data sources are fragmented and unverifiable

How PlacePro Helps

- Enforces role-based authentication — administrators, officers, and students each access only permitted functions
- Stores all data in a managed MySQL database with standard backup and recovery practices
- Provides an administrator module for user account management and system configuration
- Delivers reports generated directly from the authoritative database, improving trust in published statistics
- Establishes a governed, auditable platform that replaces ad-hoc file sharing with structured access control
- Supports deployment on institutional infrastructure with clear separation between application, database, and version-controlled source code on GitHub

---

11 11. Features

PlacePro delivers a comprehensive set of features organized around authentication, data management, student self-service, officer workflows, and institutional reporting. Each feature is numbered sequentially for traceability to functional requirements in later sections.

11.1 11.1 F1 — Student Registration

- Allows new students to create an account with validated personal and academic details (name, roll number, branch, CGPA, contact information)
- Enforces unique identifiers (e.g., roll number, email) to prevent duplicate registrations
- Captures placement-relevant attributes used later for eligibility checks (backlogs, graduation year, department)
- Stores credentials securely in MySQL with appropriate validation before account activation
- Enables the placement cell to onboard entire batches through a structured registration form rather than ad-hoc spreadsheets

11.2 11.2 F2 — Student Login

- Provides a dedicated login screen for registered students with username/password authentication
- Validates credentials against the database and establishes an authenticated session for the student role
- Redirects successful logins to the Student Dashboard; displays clear error messages on failure

- Restricts navigation to student-permitted modules only (drives, applications, profile, resume)
- Supports session persistence for the duration of application use on a shared lab machine

11.3 11.3 F3 — Admin Login

- Provides a secure login entry point for system administrators with elevated privileges
- Authenticates against administrator accounts stored separately from student and officer records
- Grants access to user management, system configuration, analytics, and full reporting modules
- Enforces stricter access boundaries — administrators can manage the platform but operational placement workflows may be delegated to officers
- Logs authentication events to support audit and troubleshooting

11.4 11.4 F4 — Placement Officer Login

- Provides a dedicated login interface for Training and Placement Officers and placement cell staff
- Authenticates officer credentials and loads the officer/admin console upon success
- Exposes modules for company management, drive management, application review, interview scheduling, and reporting
- Prevents officers from accessing system-level administrator settings reserved for the Admin role
- Ensures day-to-day placement operations are conducted through a role-appropriate interface

11.5 11.5 F5 — Recruiter Login

- Provides a restricted login portal for authorized recruiter representatives associated with registered companies
- Allows recruiters to view drives they are assigned to, review published shortlists, and confirm interview schedules
- Limits visibility to company-specific data — recruiters cannot access unrelated student records or other companies' drives
- Reduces dependency on email-based shortlist exchange by offering a read-oriented recruiter view within the system
- Supports optional outcome confirmation workflows where recruiters mark interview results for officer review

11.6 11.6 F6 — Company Management

- Enables placement officers to register and maintain recruiting company profiles (name, industry, contact person, website)
- Stores multiple job roles and package bands per company for use in drive creation
- Supports edit, deactivate, and search operations on the company master list
- Links companies to historical drive and placement records for longitudinal reporting
- Maintains a clean, standardized company directory replacing scattered contact spreadsheets

11.7 11.7 F7 — Placement Drive Management

- Allows officers to create placement drives linked to a company, job role, eligibility criteria, and visit date
- Supports drive lifecycle states: draft, published, closed, and completed
- Defines eligibility rules (minimum CGPA, allowed branches, maximum backlogs) enforced at application time
- Publishes drives to the student portal so eligible students can view and apply
- Provides officer tools to edit drive details, monitor applicant volume, and close drives when deadlines pass

11.8 11.8 F8 — Student Dashboard

- Serves as the primary landing page after student login, summarizing placement activity at a glance
- Displays counts of active drives, pending applications, shortlisted applications, and completed interviews
- Lists recently published drives and upcoming deadlines in a prioritized view
- Provides quick navigation to apply for placements, track applications, and manage profile/resume
- Reduces the need for students to navigate multiple screens to understand their current placement status

11.9 11.9 F9 — Resume Upload

- Allows students to upload a resume file (PDF or DOC format) associated with their profile
- Stores resume metadata and file path in MySQL; file stored on the local filesystem or designated upload directory
- Enables officers to access student resumes during application review and shortlisting
- Supports resume replacement when a student uploads an updated version before applying to new drives
- Validates file type and size before acceptance to prevent storage abuse

11.10 11.10 F10 — Eligibility Checker

- Evaluates a student's academic profile against a drive's configured eligibility rules before application submission
- Displays a clear eligible / not eligible result with reasons (e.g., CGPA below threshold, branch not allowed, excess backlogs)
- Prevents ineligible students from submitting applications, reducing officer workload on invalid entries
- Allows officers to preview the eligible student pool for a drive before publishing
- Operates as a shared rules engine invoked from the student apply flow and officer drive management screens

11.11 11.11 F11 — Apply for Placement

- Enables eligible students to submit a formal application to a published placement drive
- Records application timestamp, associated student, drive, and initial status (Applied)
- Prevents duplicate applications to the same drive by the same student
- Optionally attaches the student's current resume reference to the application record
- Confirms successful submission to the student with an application reference for tracking

11.12 11.12 F12 — Application Tracking

- Provides students with a detailed list of all submitted applications and their current workflow status
- Supports status values across the placement pipeline: Applied, Shortlisted, Interview Scheduled, Selected, Rejected, On Hold
- Displays status change history with timestamps so students understand progression over time
- Allows officers to update application status from the admin console, with changes reflected immediately in the student view
- Eliminates the need for students to visit the placement cell for routine status inquiries

11.13 11.13 F13 — Interview Scheduling

- Enables placement officers to schedule interview rounds for shortlisted candidates on a specific date, time, and venue
- Associates interview records with individual applications and drive details
- Displays scheduled interviews to students through the Application Tracking and Dashboard modules
- Supports multiple rounds per application (e.g., aptitude, technical, HR) with round-wise outcome recording

- Provides officers with a schedule view filtered by drive, date, or company for on-campus coordination

11.14 11.14 F14 — Notifications

- Delivers in-application alerts to users when placement-relevant events occur (new drive published, status change, interview scheduled)
- Maintains a notification inbox within the desktop client accessible from the user dashboard
- Marks notifications as read/unread and displays unread counts for at-a-glance awareness
- Targets notifications by role — students receive student-facing alerts; officers receive administrative alerts
- Provides timely awareness without requiring external email or SMS integration in v1.0

11.15 11.15 F15 — Reports

- Generates structured placement reports exportable for institutional review and accreditation documentation
- Supports report types including: placement summary by department, company-wise selection statistics, drive-wise applicant funnel, and individual student placement record
- Allows filtering by academic year, department, company, and date range
- Outputs reports in printable or exportable format (PDF/CSV) from the desktop client
- Replaces end-of-season manual spreadsheet consolidation with database-driven report generation

11.16 11.16 F16 — Analytics Dashboard

- Presents visual summaries of placement performance metrics for administrators and officers
- Displays charts and KPIs: total placements, placement percentage by department, average package, top recruiting companies, and application-to-selection conversion rates
- Updates dynamically from live MySQL data rather than static exports
- Supports drill-down from high-level metrics to underlying drive and application detail
- Enables leadership to monitor placement season progress in real time without requesting custom reports

11.17 11.17 F17 — Student Search

- Provides officers and administrators with a searchable index of all registered students
- Supports search and filter by name, roll number, branch, CGPA range, and placement status
- Returns results in a paginated table with quick links to student profile, applications, and resume
- Accelerates shortlist preparation and individual student lookup during active drives
- Reduces time spent scrolling through master Excel sheets to find specific records

11.18 11.18 F18 — Company Search

- Provides a searchable directory of all registered recruiting companies and their associated drives
- Supports filter by company name, industry, active drive status, and historical placement count
- Enables officers to quickly locate company records for editing, drive creation, or report generation
- Displays linked drives and placement outcomes per company for relationship management
- Complements Company Management (F6) by optimizing discovery and lookup of existing records

11.19 11.19 F19 — Logout

- Provides a consistent logout action available from all role-based interfaces (Student, Officer, Admin, Recruiter)
- Terminates the active user session and clears role-specific navigation state from the client
- Redirects the user to the appropriate login selection screen after logout
- Prevents session reuse on shared lab machines — critical for security in college computer labs
- Ensures clean session boundaries between consecutive users on the same workstation

11.20 11.20 Feature Overlap and Consolidation Notes

The following overlaps were identified during feature review. They are documented here for clarity during functional requirements mapping and implementation planning. No features have been removed; overlaps indicate shared logic or UI surfaces that should be implemented once and reused.

Overlap	Features Involved	Nature of Overlap	Recommendation
Authentication cluster	F2, F3, F4, F5, F19	Four separate login entry points plus logout are all part of a single authentication and session-management subsystem	Implement a unified auth module with role-based routing; separate login screens may remain for UX clarity
Officer vs. Admin access	F3, F4	Admin Login and Placement Officer Login both grant access to management consoles with partially shared modules (reports, search, drives)	Define a clear permission matrix — officers handle operations; admins handle users, config, and analytics
Eligibility and application	F10, F11	Eligibility Checker logic is invoked as a prerequisite step within Apply for Placement	Implement F10 as a shared service called by F11; avoid duplicating eligibility rules in two places
Dashboard vs. tracking	F8, F12	Student Dashboard summarizes application status; Application Tracking provides the detailed list — both surface the same underlying application data	Dashboard reads aggregate counts and recent items; F12 owns the detailed status view and history
Reports vs. analytics	F15, F16	Reports (tabular/export) and Analytics Dashboard (visual KPIs) draw from the same placement dataset and may show redundant metrics	Share a common reporting data layer; F15 focuses on exportable documents, F16 on live visual summaries
Search vs. management	F6, F18	Company Search overlaps with browse/filter functionality within Company Management	F18 provides global quick-search; F6 remains the CRUD interface — share the same company query service
Search vs. management	F1, F17	Student Search overlaps with locating students already registered via Student Registration	F17 is an officer/admin lookup tool; F1 is the onboarding entry point — share the student data model
Notifications vs. status updates	F14, F12	Status changes in Application Tracking may trigger Notifications; content can feel redundant if both show the same event	Notifications should alert users to changes; F12 remains the authoritative status record
Recruiter role scope	F5, F4	Recruiter Login (F5) and officer workflows (F4) both touch shortlists and interview scheduling	Define clear handoff: officers manage shortlists; recruiters view schedules and record outcomes within company-scoped access
Notifications scope	F14, Scope §7.2	In-app notifications (F14) are in scope; external email/SMS remain out of scope (Section 7.2, Section 19)	Implement F14 as a desktop-client notification inbox only; no external delivery in v1.0

Summary: No duplicate feature IDs were assigned. The most significant consolidation opportunities are **authentication (F2–F5, F19)**, **reports/analytics (F15–F16)**, and **eligibility/apply (F10–F11)**. Implementation should favor shared services behind distinct user-facing features to maintain PRD traceability without code duplication.

12 12. Use Flow

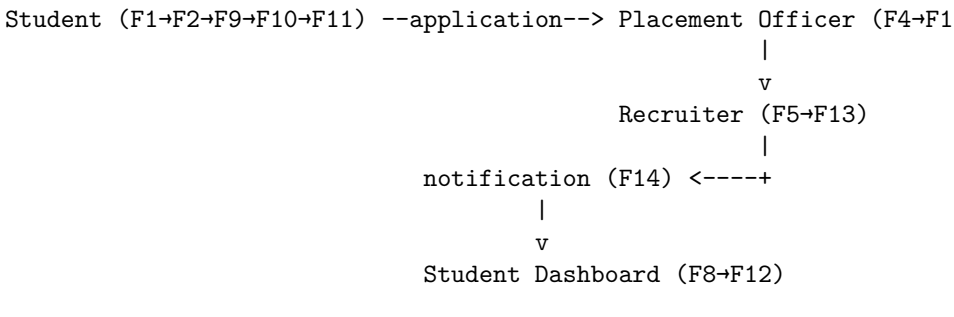
This section describes the primary and secondary end-to-end workflows in PlacePro. Each step is numbered sequentially within its flow and references the feature IDs defined in Section 11 where applicable.

12.1 12.1 Primary Flow — Student Placement Journey

Scenario: A final-year student registers on PlacePro, applies to a published drive, is shortlisted by the placement officer, receives an interview schedule from the recruiter, and is notified of the outcome.

Step	Action	Feature(s)
1	**Register account** — Student launches PlacePro and opens the Student Registration screen. Student enters personal and academic details (name, roll number, branch, CGPA, email, contact number) and sets a password. System validates uniqueness of roll number and email, then creates the student account in MySQL.	F1
2	**Log in** — Student navigates to the Student Login screen, enters credentials, and authenticates. System validates credentials, establishes a student session, and redirects to the Student Dashboard.	F2, F8
3	**Upload resume** — From the dashboard or profile section, student selects a resume file (PDF/DOC) and uploads it. System validates file type and size, stores the file, and links the resume reference to the student profile.	F9
4	**Browse drives** — Student views the list of published placement drives on the dashboard. Student selects a drive of interest to view company name, job role, package, eligibility criteria, and application deadline.	F8, F7
5	**Check eligibility** — Student initiates an eligibility check for the selected drive. System evaluates the student's CGPA, branch, and backlog count against the drive's configured rules and displays an eligible or not eligible result with explanatory reasons.	F10
6	**Apply for placement** — If eligible, student confirms application submission. System records the application with status *Applied*, timestamps the submission, attaches the resume reference, and prevents duplicate applications to the same drive.	F11
7	**Officer reviews application** — Placement Officer logs in via the Placement Officer Login screen and navigates to the drive's application list. Officer reviews the student's profile, academic details, and resume, then updates the application status to *Shortlisted* or *Rejected*.	F4, F12, F9
8	**Recruiter schedules interview** — Recruiter logs in via the Recruiter Login portal and views the shortlisted candidates for the assigned drive. Recruiter selects a candidate and proposes an interview date, time, and venue (or confirms a schedule coordinated with the officer). System creates an interview record linked to the application.	F5, F13
9	**Officer confirms schedule** — Placement Officer reviews the recruiter's proposed interview schedule, confirms or adjusts details, and finalizes the interview round in the system. Application status updates to *Interview Scheduled*.	F4, F13, F12
10	**Student receives notification** — System generates an in-app notification for the student indicating that an interview has been scheduled. Student sees an unread notification badge on the Student Dashboard, opens the notification inbox, and views interview date, time, venue, and company details. Student also sees the updated status in Application Tracking.	F14, F8, F12
11	**Attend interview and receive outcome** — Student attends the scheduled interview. Recruiter or officer records the round outcome (Selected / Rejected / On Hold) in the system. Student receives a further notification of the final status and views the result in Application Tracking.	F13, F14, F12
12	**Log out** — Student clicks Logout from the dashboard. System terminates the session and returns to the login screen, securing the workstation for the next user.	F19

Primary Flow Diagram (Logical)



12.2 12.2 Secondary Flow — Administrative Setup and Recruiter Status Updates

Scenario: An administrator onboards a recruiting company and creates a placement drive before the season opens. A placement officer manages incoming applications during the drive. A recruiter logs in to update interview outcomes after on-campus rounds.

Step	Action	Feature(s)
1	**Admin logs in** — Administrator opens PlacePro and authenticates via the Admin Login screen. System grants access to the administrator console with user management, company, drive, and analytics modules.	F3
2	**Add company** — Administrator navigates to Company Management and creates a new company profile (company name, industry, contact person, email, phone). System stores the record in MySQL and displays it in the company directory.	F6
3	**Verify company record** — Administrator uses Company Search to locate the newly added company and confirms details are accurate before drive creation.	F18, F6
4	**Create placement drive** — Administrator opens Placement Drive Management, selects the registered company, and configures a new drive: job role, package, eligibility criteria (minimum CGPA, allowed branches, max backlogs), visit date, and application deadline. Drive is saved in *Draft* state.	F7
5	**Publish drive** — Administrator reviews drive configuration, changes status from *Draft* to *Published*, and makes the drive visible on the student portal. Eligible students can now discover and apply to the drive.	F7, F10
6	**Officer logs in** — Placement Officer authenticates via the Placement Officer Login screen and accesses the officer console for day-to-day application management.	F4
7	**Manage applications** — Officer opens the published drive's application queue. Officer filters applications by status, uses Student Search to locate specific candidates if needed, reviews profiles and resumes, and bulk-updates statuses (Shortlisted / Rejected). Officer monitors applicant volume via the Analytics Dashboard.	F12, F17, F9, F16
8	**Generate shortlist for recruiter** — Officer exports or publishes the finalized shortlist associated with the drive. Recruiter account is linked to the company and can now view assigned candidates upon login.	F12, F5, F15
9	**Recruiter logs in** — Recruiter authenticates via the Recruiter Login portal. System displays only drives and shortlists associated with the recruiter's company.	F5
10	**Recruiter updates interview status** — After conducting on-campus interviews, recruiter opens the interview schedule for the drive, selects each candidate, and records round-wise outcomes (Selected / Rejected / On Hold). System updates application status and triggers in-app notifications to affected students.	F5, F13, F12, F14
11	**Officer reviews outcomes** — Placement Officer verifies recruiter-submitted outcomes, resolves any discrepancies, and finalizes placement records. Officer generates a drive-wise placement report for institutional records.	F4, F12, F15

Step	Action	Feature(s)
12	**Admin reviews analytics** — Administrator opens the Analytics Dashboard to review placement metrics for the drive (applications received, shortlist rate, selection rate, department breakdown) and confirms data accuracy for leadership reporting.	F3, F16, F15
13	**Log out** — Administrator, officer, or recruiter terminates their session via Logout, returning the client to the login screen.	F19

Secondary Flow Diagram (Logical)

```
Admin (F3→F6→F7) --publish drive--> Student Portal
                                          |
                                          v
                                Placement Officer (F4→F12→F17)
                                          |
                                          v
                                    Recruiter (F5→F13→F14)
                                          |
                                          v
                                Officer / Admin Reports (F15→F16)
```

12.3 12.3 Flow-to-Feature Reference Summary

Flow Step (User Request)	PRD Step(s)	Feature ID(s)
Student registers	Primary 1	F1
Student logs in	Primary 2	F2, F8
Student uploads resume	Primary 3	F9
Student checks eligibility	Primary 5	F10
Student applies	Primary 6	F11
Placement Officer reviews	Primary 7	F4, F12
Recruiter schedules interview	Primary 8–9	F5, F13
Student receives notification	Primary 10	F14, F12
Admin adds companies	Secondary 2–3	F3, F6, F18
Admin creates placement drives	Secondary 4–5	F7
Placement Officer manages applications	Secondary 7–8	F4, F12, F17
Recruiter updates interview status	Secondary 10	F5, F13, F14

---

13 13. Functional Requirements

The following functional requirements define the mandatory behaviors PlacePro shall exhibit. Each requirement is traceable to a feature defined in Section 11. Requirements are phrased in industry-standard *shall* statement format and assigned a priority for implementation planning.

Priority definitions:

Priority	Definition
High	Core functionality; system cannot meet primary use flows without this requirement

Priority	Definition
Medium	Important supporting functionality; degrades user experience if omitted but system remains usable
Low	Desirable enhancement; may be deferred without blocking primary placement workflows

13.1 13.1 Requirements Table

ID	Feature	Requirement	Priority
FR-01	F1 — Student Registration	The system shall allow new students to register by providing name, roll number, branch, CGPA, email, contact number, and password.	High
FR-02	F1 — Student Registration	The system shall reject registration attempts when the roll number or email address already exists in the database.	High
FR-03	F1 — Student Registration	The system shall validate all mandatory registration fields and display descriptive error messages before persisting a student record.	High
FR-04	F2 — Student Login	The system shall authenticate students using username and password credentials stored in MySQL and establish a student-role session upon success.	High
FR-05	F2 — Student Login	The system shall redirect authenticated students to the Student Dashboard and deny access to administrator and officer management modules.	High
FR-06	F3 — Admin Login	The system shall authenticate administrator credentials and grant access to user management, system configuration, analytics, and reporting modules.	High
FR-07	F3 — Admin Login	The system shall allow administrators to create, deactivate, and reset user accounts for student, officer, and recruiter roles.	High
FR-08	F4 — Placement Officer Login	The system shall authenticate placement officer credentials and grant access to company management, drive management, application review, and interview scheduling modules.	High
FR-09	F4 — Placement Officer Login	The system shall prevent placement officers from modifying system-level administrator settings reserved for the Admin role.	Medium
FR-10	F5 — Recruiter Login	The system shall authenticate recruiter credentials and restrict data visibility to drives and shortlists associated with the recruiter's assigned company.	Medium
FR-11	F5 — Recruiter Login	The system shall allow recruiters to view shortlisted candidate profiles and interview schedules for assigned drives only.	Medium
FR-12	F6 — Company Management	The system shall allow authorized users to create, edit, and deactivate company profiles with name, industry, contact person, and contact details.	High
FR-13	F6 — Company Management	The system shall persist company records in MySQL and associate each company with its placement drives and historical placement outcomes.	High
FR-14	F7 — Placement Drive Management	The system shall allow authorized users to create placement drives linked to a company, job role, package, eligibility criteria, visit date, and application deadline.	High

ID	Feature	Requirement	Priority
FR-15	F7 — Placement Drive Management	The system shall support drive lifecycle states — Draft, Published, Closed, and Completed — and enforce valid state transitions.	High
FR-16	F7 — Placement Drive Management	The system shall display published drives to eligible students on the student portal and prevent applications to closed or draft drives.	High
FR-17	F8 — Student Dashboard	The system shall display a summary of active drives, application counts by status, and upcoming deadlines on the Student Dashboard after login.	High
FR-18	F8 — Student Dashboard	The system shall provide navigation shortcuts from the Student Dashboard to drive listings, application tracking, profile management, and resume upload.	Medium
FR-19	F9 — Resume Upload	The system shall allow students to upload resume files in PDF or DOC format and validate file type and maximum file size before acceptance.	High
FR-20	F9 — Resume Upload	The system shall store the resume file reference in MySQL and associate it with the student profile and subsequent application submissions.	High
FR-21	F10 — Eligibility Checker	The system shall evaluate a student's eligibility against a drive's configured rules (minimum CGPA, allowed branches, maximum backlogs) and return an eligible or ineligible result.	High
FR-22	F10 — Eligibility Checker	The system shall display specific ineligibility reasons when a student fails eligibility validation for a selected drive.	Medium
FR-23	F11 — Apply for Placement	The system shall allow eligible students to submit an application to a published drive and record the application with status *Applied* and a submission timestamp.	High
FR-24	F11 — Apply for Placement	The system shall prevent a student from submitting more than one application to the same placement drive.	High
FR-25	F12 — Application Tracking	The system shall maintain application records with workflow statuses: Applied, Shortlisted, Interview Scheduled, Selected, Rejected, and On Hold.	High
FR-26	F12 — Application Tracking	The system shall allow placement officers to update application status and reflect changes immediately in the student's Application Tracking view.	High
FR-27	F12 — Application Tracking	The system shall record a timestamped status history for each application state transition.	Medium
FR-28	F13 — Interview Scheduling	The system shall allow placement officers and recruiters to schedule interview rounds with date, time, venue, and round type linked to a specific application.	High
FR-29	F13 — Interview Scheduling	The system shall support multiple interview rounds per application and allow recording of round-wise outcomes (Selected, Rejected, On Hold).	Medium
FR-30	F14 — Notifications	The system shall generate in-app notifications when a new drive is published, an application status changes, or an interview is scheduled.	Medium
FR-31	F14 — Notifications	The system shall maintain a per-user notification inbox with read/unread indicators and an unread count visible on the user dashboard.	Medium

ID	Feature	Requirement	Priority
FR-32	F15 — Reports	The system shall generate placement reports filterable by academic year, department, company, and drive, including placement summary and drive-wise applicant statistics.	High
FR-33	F15 — Reports	The system shall export generated reports in PDF or CSV format from the desktop client.	Medium
FR-34	F16 — Analytics Dashboard	The system shall display visual analytics including total placements, placement percentage by department, average package, and application-to-selection conversion rate.	Medium
FR-35	F16 — Analytics Dashboard	The system shall compute analytics metrics dynamically from live MySQL data without requiring manual data export.	Medium
FR-36	F17 — Student Search	The system shall allow placement officers and administrators to search students by name, roll number, branch, CGPA range, and placement status.	Medium
FR-37	F18 — Company Search	The system shall allow placement officers and administrators to search companies by name, industry, and active drive status.	Medium
FR-38	F19 — Logout	The system shall terminate the active user session, clear role-specific client state, and redirect the user to the login screen upon logout.	High
FR-39	F19 — Logout	The system shall provide a logout action accessible from all role-based interfaces (Student, Officer, Admin, Recruiter).	High

13.2 13.2 Requirements Summary by Priority

Priority	Count	Requirement IDs
High	25	FR-01 – FR-08, FR-12 – FR-17, FR-19 – FR-21, FR-23 – FR-26, FR-28, FR-32, FR-38, FR-39
Medium	14	FR-09 – FR-11, FR-18, FR-22, FR-27, FR-29 – FR-31, FR-33 – FR-37
Low	0	—

13.3 13.3 Feature Traceability Matrix

Feature	Requirement IDs
F1 — Student Registration	FR-01, FR-02, FR-03
F2 — Student Login	FR-04, FR-05
F3 — Admin Login	FR-06, FR-07
F4 — Placement Officer Login	FR-08, FR-09
F5 — Recruiter Login	FR-10, FR-11
F6 — Company Management	FR-12, FR-13
F7 — Placement Drive Management	FR-14, FR-15, FR-16
F8 — Student Dashboard	FR-17, FR-18
F9 — Resume Upload	FR-19, FR-20
F10 — Eligibility Checker	FR-21, FR-22
F11 — Apply for Placement	FR-23, FR-24
F12 — Application Tracking	FR-25, FR-26, FR-27
F13 — Interview Scheduling	FR-28, FR-29
F14 — Notifications	FR-30, FR-31
F15 — Reports	FR-32, FR-33
F16 — Analytics Dashboard	FR-34, FR-35

Feature	Requirement IDs
F17 — Student Search	FR-36
F18 — Company Search	FR-37
F19 — Logout	FR-38, FR-39

Total functional requirements: 39 (FR-01 through FR-39)

---

14 14. Non-Functional Requirements

Non-functional requirements define the quality attributes, operational constraints, and system-wide behaviors that PlacePro must satisfy alongside its functional capabilities. Each requirement is numbered sequentially (NFR-01 onward) and grouped by category for implementation and testing reference.

14.1 14.1 Requirements Summary Table

ID	Category	Requirement
NFR-01	Performance	Standard CRUD operations (create, read, update, delete) shall complete within 3 seconds on a college lab machine with up to 500 student records.
NFR-02	Performance	Report generation and analytics dashboard loading shall complete within 5 seconds for datasets covering a full placement season (up to 50 drives, 500 students).
NFR-03	Performance	Student search and company search queries shall return results within 2 seconds for indexed lookups on the full student and company tables.
NFR-04	Security	All user passwords shall be hashed using a one-way hashing algorithm (e.g., BCrypt) with a minimum work factor of 10 before storage in MySQL.
NFR-05	Security	The system shall enforce role-based access control (RBAC) so that students, officers, recruiters, and administrators can access only modules and data permitted by their role.
NFR-06	Security	The system shall lock out or throttle repeated failed login attempts (minimum: 5 consecutive failures within 10 minutes) to mitigate brute-force attacks.
NFR-07	Security	Database connection credentials shall be stored in a configuration file excluded from version control (`.gitignore`); no plaintext passwords shall be hardcoded in source code.
NFR-08	Reliability	The system shall maintain data integrity through transactional JDBC operations for all multi-step writes (e.g., application submission with status history).
NFR-09	Reliability	The application shall recover gracefully from unexpected shutdown without corrupting the MySQL database schema or leaving partial records in an inconsistent state.
NFR-10	Availability	The system shall remain operational during normal college lab hours (minimum 8 hours continuous use) without requiring application restart under expected load.
NFR-11	Availability	MySQL connection failures shall be detected on startup and during runtime; the application shall display a clear message and retry connection rather than crashing silently.
NFR-12	Usability	A first-time student shall be able to register, log in, and submit an application to a published drive within 5 minutes without external documentation.
NFR-13	Usability	A first-time placement officer shall be able to create a company, publish a drive, and review an application within 10 minutes without external documentation.
NFR-14	Usability	All user-facing error and validation messages shall be written in plain language and shall indicate the corrective action where applicable.
NFR-15	Maintainability	The codebase shall follow a layered architecture separating presentation (Swing UI), business logic (service layer), and data access (DAO layer) into distinct packages.
NFR-16	Maintainability	All source code shall be version-controlled using Git with a hosted repository on GitHub, including meaningful commit messages and branch history.

ID	Category	Requirement
NFR-17	Maintainability	Database schema changes shall be documented via SQL migration scripts stored in the repository to support reproducible setup across development machines.
NFR-18	Scalability	The system shall support a minimum of 500 registered student records and 50 concurrent placement drives without measurable degradation in CRUD response times.
NFR-19	Scalability	The database schema shall be normalized to third normal form (3NF) to minimize redundancy and support growth in companies, drives, and application records.
NFR-20	Portability	The application shall run on any desktop operating system supporting Java Runtime Environment (JRE) 11 or higher (Windows, macOS, Linux).
NFR-21	Portability	The system shall connect to MySQL 8.0 or higher via JDBC without vendor-specific database extensions that would prevent migration to another relational database.
NFR-22	Database	All persistent data shall be stored in a MySQL relational database accessed exclusively through JDBC; no business data shall be stored in flat files except uploaded resume documents.
NFR-23	Database	The database schema shall enforce referential integrity through primary keys and foreign keys linking students, companies, drives, applications, interviews, and notifications.
NFR-24	Database	All database queries shall use parameterized PreparedStatements to prevent SQL injection vulnerabilities.
NFR-25	Backup	The system documentation shall include instructions for performing a full MySQL database backup using `mysqldump` or equivalent before each placement season.
NFR-26	Backup	Resume upload files shall be stored in a designated directory structure that can be backed up independently of the database, with file paths referenced in MySQL.
NFR-27	Logging	The system shall write application logs to a local log file capturing authentication events, database errors, and unhandled exceptions.
NFR-28	Logging	Log entries shall include a timestamp, severity level (INFO, WARN, ERROR), and a descriptive message sufficient for post-incident troubleshooting.
NFR-29	Error Handling	The application shall catch and handle all SQLException instances at the DAO layer and propagate user-friendly messages to the UI without exposing raw SQL or stack traces.
NFR-30	Error Handling	The application shall validate all user input at the UI layer before submission and display field-level validation errors without losing previously entered data.
NFR-31	Error Handling	Unhandled exceptions at the UI layer shall be caught by a global exception handler that logs the error and displays a generic recovery message to the user.
NFR-32	Response Time	The Student Dashboard shall load and display summary data within 2 seconds of successful student login.
NFR-33	Response Time	Eligibility check results shall be displayed within 1 second of the student initiating a check against a selected drive.
NFR-34	Response Time	Application status updates performed by an officer shall be visible to the affected student within 2 seconds of the officer saving the status change (on next screen refresh or notification poll).

14.2 14.2 Performance

ID	Requirement
NFR-01	Standard CRUD operations shall complete within **3 seconds** with up to 500 student records.
NFR-02	Report generation and analytics loading shall complete within **5 seconds** for a full placement season dataset.

ID	Requirement
NFR-03	Search queries shall return results within **2 seconds** for full-table indexed lookups.

14.3 14.3 Security

ID	Requirement
NFR-04	Passwords shall be hashed with BCrypt (work factor >= 10) before MySQL storage.
NFR-05	Role-based access control shall restrict modules and data by user role.
NFR-06	Failed login attempts shall be throttled after 5 consecutive failures within 10 minutes.
NFR-07	Database credentials shall reside in a `.gitignore`-excluded config file; no hardcoded secrets in source.

14.4 14.4 Reliability

ID	Requirement
NFR-08	Multi-step database writes shall use transactional JDBC operations to ensure atomicity.
NFR-09	Unexpected application shutdown shall not corrupt schema or leave inconsistent partial records.

14.5 14.5 Availability

ID	Requirement
NFR-10	The application shall sustain **8+ hours** of continuous operation without restart under expected load.
NFR-11	MySQL connection failures shall be detected and reported with retry capability rather than silent crash.

14.6 14.6 Usability

ID	Requirement
NFR-12	First-time students shall complete registration through application submission within **5 minutes** unaided.
NFR-13	First-time officers shall create a company, publish a drive, and review an application within **10 minutes** unaided.

ID	Requirement
NFR-14	All error messages shall use plain language with corrective guidance where applicable.

14.7 14.7 Maintainability

ID	Requirement
NFR-15	Codebase shall follow layered architecture: UI → Service → DAO.
NFR-16	Source code shall be version-controlled on GitHub with meaningful commit history.
NFR-17	Database schema changes shall be documented via SQL migration scripts in the repository.

14.8 14.8 Scalability

ID	Requirement
NFR-18	System shall support **500 students** and **50 active drives** without CRUD degradation.
NFR-19	Database schema shall be normalized to **3NF** to support record growth without redundancy.

14.9 14.9 Portability

ID	Requirement
NFR-20	Application shall run on Windows, macOS, and Linux with **JRE 11+**.
NFR-21	Database access shall use standard JDBC with **MySQL 8.0+**; no vendor-locking extensions.

14.10 14.10 Database

ID	Requirement
NFR-22	All business data shall persist in MySQL via JDBC; resumes are the only permitted file-based data.
NFR-23	Schema shall enforce referential integrity with primary and foreign keys across all entities.
NFR-24	All SQL queries shall use **parameterized PreparedStatements** to prevent injection.

14.11 14.11 Backup

ID	Requirement
NFR-25	Documentation shall include `mysqldump` backup procedures to be executed before each placement season.
NFR-26	Resume files shall reside in a separately backable directory with paths stored in MySQL.

14.12 14.12 Logging

ID	Requirement
NFR-27	Application shall write logs for authentication events, database errors, and unhandled exceptions.
NFR-28	Each log entry shall include timestamp, severity level, and descriptive message.

14.13 14.13 Error Handling

ID	Requirement
NFR-29	SQLException instances shall be caught at the DAO layer; UI shall show user-friendly messages only.
NFR-30	UI input validation shall occur before submission with field-level errors and data preservation.
NFR-31	A global UI exception handler shall log errors and display a generic recovery message.

14.14 14.14 Response Time

ID	Requirement
NFR-32	Student Dashboard shall load within **2 seconds** after login.
NFR-33	Eligibility check results shall display within **1 second**.
NFR-34	Officer status updates shall be visible to students within **2 seconds** of save.

14.15 14.15 Category Summary

Category	Requirement IDs	Count
Performance	NFR-01 – NFR-03	3
Security	NFR-04 – NFR-07	4
Reliability	NFR-08 – NFR-09	2
Availability	NFR-10 – NFR-11	2
Usability	NFR-12 – NFR-14	3
Maintainability	NFR-15 – NFR-17	3
Scalability	NFR-18 – NFR-19	2
Portability	NFR-20 – NFR-21	2
Database	NFR-22 – NFR-24	3
Backup	NFR-25 – NFR-26	2
Logging	NFR-27 – NFR-28	2
Error Handling	NFR-29 – NFR-31	3
Response Time	NFR-32 – NFR-34	3

Total non-functional requirements: 34 (NFR-01 through NFR-34)

---

15 15. Technology Stack

PlacePro is built as a standalone desktop application. The technology stack is deliberately chosen to support offline-capable, local-network deployment within a college placement cell — without dependency on web servers, cloud services, or browser-based frameworks. Each component serves a defined role in the presentation, persistence, development, and delivery of the system.

15.1 15.1 Technology Stack Overview

Layer	Technology	Purpose
Programming Language	Java	Core application logic, business rules, and desktop runtime
Graphical User Interface	Java Swing	Native desktop forms, tables, dialogs, and navigation for all user roles
Database	MySQL	Relational persistence for students, companies, drives, applications, and reports
Database Connectivity	JDBC	Standard Java API for connecting the application to MySQL
Integrated Development Environment	IntelliJ IDEA	Project development, debugging, refactoring, and build management
Version Control	Git	Local source history, branching, and change tracking
Repository Hosting	GitHub	Remote repository for collaboration, backup, and academic submission

---

15.2 15.2 Technology Selection Rationale

15.2.1 15.2.1 Java

Java was selected as the primary programming language because it is the industry-standard platform for enterprise desktop and business applications. For a placement management system that must handle structured data, role-based workflows, and concurrent access from multiple lab machines, Java provides:

- **Platform independence** — The application runs on Windows, macOS, and Linux lab machines with a single JRE installation, matching the heterogeneous environments found in college computer labs.

- **Mature ecosystem** — Extensive libraries, documentation, and community support reduce development risk for an academic project with a fixed timeline.
- **Strong typing and OOP** — Object-oriented design maps naturally to placement domain entities (Student, Company, Drive, Application), supporting the layered architecture required by NFR-15.
- **Academic alignment** — Java is a core component of most computer science curricula, making the codebase understandable to evaluators and future student maintainers.

Java is used exclusively as a desktop runtime. No web application server or servlet container is required.

15.2.2 15.2.2 Java Swing

Java Swing was selected as the GUI framework because PlacePro is a **standalone desktop application** intended for installation on placement cell workstations and student lab machines — not a browser-based portal.

- **Native desktop experience** — Swing provides form-based screens, data tables, modal dialogs, and menu navigation suited to administrative data-entry workflows (company registration, drive creation, application review).
- **Zero deployment overhead** — No web server, no browser compatibility matrix, and no frontend build toolchain. The application launches as a single desktop process.
- **Bundled with JRE** — Swing is included in the standard Java distribution; no additional GUI dependencies are required.
- **Table and form components** — `JTable`, `JForm`, `JDialog`, and `JTabbedPane` directly support the searchable student lists, application queues, and multi-panel dashboards described in the feature set.
- **Offline operation** — Core placement workflows function on a local network without internet connectivity, critical for institutions with limited or unreliable external network access during placement season.

Swing was chosen over JavaFX for broader academic familiarity and simpler project setup; both remain valid desktop-only options with no web stack involved.

15.2.3 15.2.3 MySQL

MySQL was selected as the relational database because placement management is inherently relational — students apply to drives, drives belong to companies, applications have status histories, and interviews link to applications.

- **Relational integrity** — Foreign keys enforce valid relationships between entities (NFR-23), preventing orphaned application records or references to deleted drives.
- **Structured querying** — SQL supports the search, filter, report, and analytics requirements (F15, F16, F17, F18) with efficient joins across normalized tables.
- **College lab familiarity** — MySQL is widely available in academic environments, commonly preinstalled or easily deployed on a local server within the placement cell network.
- **Proven scalability for scope** — MySQL comfortably handles the projected data volume (500 students, 50 drives, thousands of application records) defined in NFR-18 without requiring enterprise database licensing.
- **Backup tooling** — Standard `mysqldump` utilities support the backup procedures required by NFR-25.

15.2.4 15.2.4 JDBC

JDBC (Java Database Connectivity) was selected as the database access layer because it is the native, standard Java API for relational database interaction — providing a direct, transparent connection between the Swing application and MySQL without an intermediate framework.

- **No middleware dependency** — JDBC connects the desktop client directly to MySQL on the local network. No application server, ORM container, or web API layer sits between the UI and the database.

- **Explicit SQL control** — DAO classes write parameterized SQL queries (NFR-24), giving full visibility over database operations — valuable for academic evaluation and debugging.
- **Transactional support** — `Connection.setAutoCommit(false)` with commit/rollback enables atomic multi-step operations such as application submission with status history (NFR-08).
- **Driver portability** — The MySQL Connector/J JDBC driver is a single JAR dependency, keeping the project lightweight and self-contained.
- **Curriculum alignment** — JDBC is a standard topic in Java and database courses, demonstrating practical database programming skills.

15.2.5 15.2.5 IntelliJ IDEA

IntelliJ IDEA was selected as the integrated development environment for efficient development of a multi-package Java desktop project.

- **Project structure management** — Supports layered package organization (UI, service, DAO, model, util) required by the maintainability standards in NFR-15.
- **Integrated debugging** — Breakpoint debugging across Swing event handlers and JDBC calls accelerates troubleshooting of form validation and database errors.
- **Database tools** — Built-in database console allows developers to inspect MySQL schema, run test queries, and validate data during development without leaving the IDE.
- **Refactoring support** — Automated rename, extract method, and dependency analysis reduce errors when evolving the codebase across development sprints.
- **Build and run** — One-click compile and launch of the Swing application streamlines the demo and testing cycle during placement season development.

15.2.6 15.2.6 Git

Git was selected for distributed version control to manage source code history throughout the academic project lifecycle.

- **Change tracking** — Every feature, bug fix, and schema update is recorded with authorship and timestamps, supporting academic integrity and project review.
- **Branching** — Feature branches (e.g., `feature/student-registration`, `feature/drive-management`) allow parallel development without destabilizing the main codebase.
- **Rollback capability** — Failed experiments or breaking changes can be reverted to a known-good commit, reducing risk during iterative development.
- **Collaboration** — Multiple team members can work on separate modules (UI, DAO, reports) and merge changes through standard pull workflows.

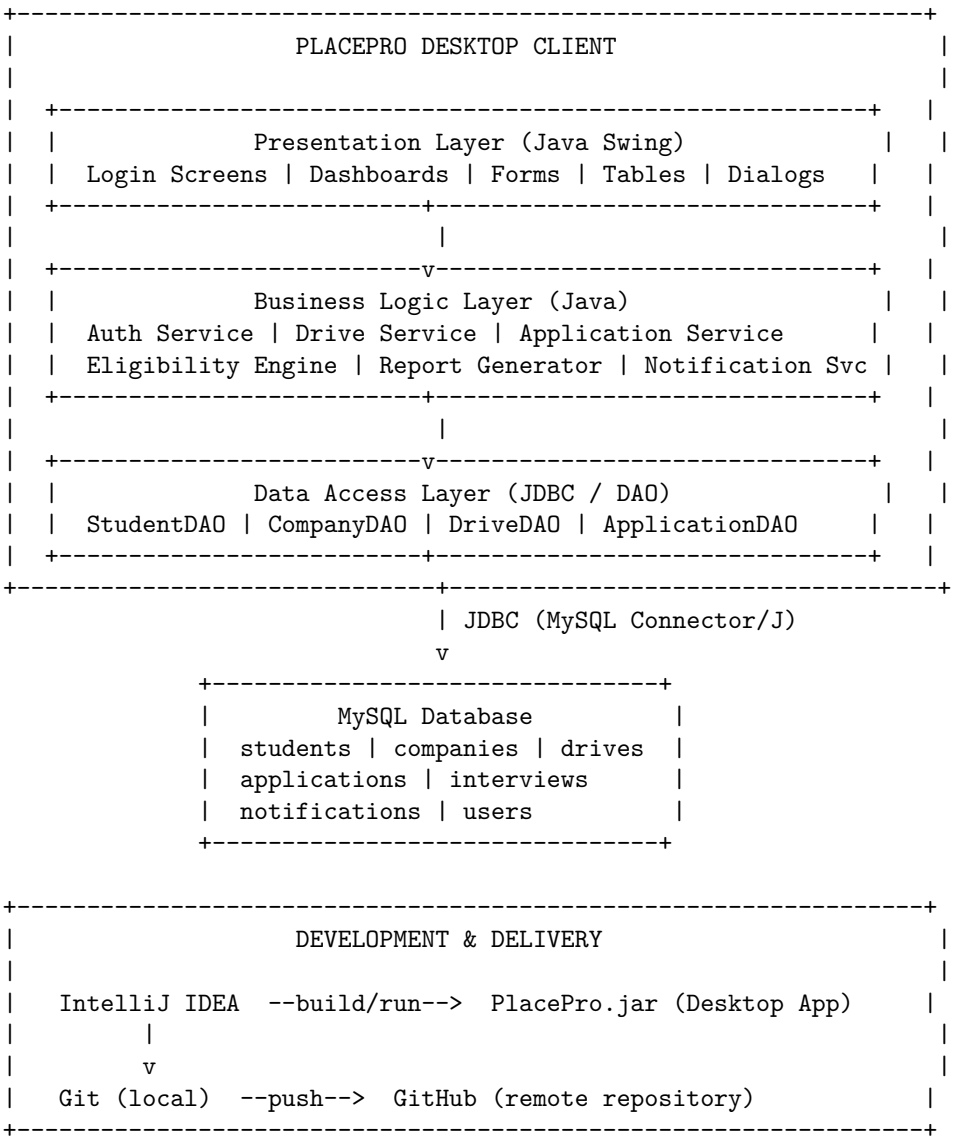
15.2.7 15.2.7 GitHub

GitHub was selected as the remote repository hosting platform to complement local Git version control.

- **Remote backup** — Source code is protected against local machine failure, aligning with institutional expectations for project deliverable preservation.
- **Academic submission** — Evaluators can review the repository, commit history, README, and project structure directly via a shareable URL.
- **Issue tracking** — GitHub Issues supports task assignment, bug reporting, and milestone tracking across the development team.
- **Documentation hosting** — Repository README, wiki, and SQL migration scripts (NFR-17) are accessible alongside the codebase for setup and evaluation.
- **Portfolio value** — A well-maintained GitHub repository demonstrates professional development practices to recruiters and future employers.

15.3 15.3 Architecture Diagram (Logical)

PlacePro follows a three-tier desktop architecture. All tiers run on the local machine or local network — there is no web server, no browser client, and no cloud component.



15.4 15.4 Technology Constraints

The following constraints apply to the PlacePro technology stack for v1.0:

Constraint	Detail
Desktop only	No web frameworks, no browser-based UI, no application server
Java runtime	JRE 11 or higher required on all deployment machines
Database server	MySQL 8.0 or higher on local machine or campus LAN
Network	Local network sufficient; internet required only for GitHub push/pull
Resume storage	Local filesystem directory; paths referenced in MySQL (not BLOB storage)

Constraint	Detail
Build output	Executable JAR launched from IntelliJ IDEA or command line (`java -jar PlacePro.jar`)

15.5 15.5 Dependency Summary

Dependency	Type	Notes
Java SE 11+	Runtime	Core language and Swing UI
MySQL Connector/J	JDBC Driver	Maven/Gradle dependency or standalone JAR
MySQL Server 8.0+	External service	Installed separately on deployment machine
IntelliJ IDEA	Development tool	Community or Ultimate edition
Git	Version control	Command-line or IDE-integrated
GitHub	Remote hosting	Free public or private repository

Explicitly excluded from v1.0: Web frameworks, servlet containers, cloud databases, ORM frameworks, and browser-based frontends. PlacePro is a self-contained Java Swing desktop application connecting to MySQL via JDBC.

---

16 16. Database Design

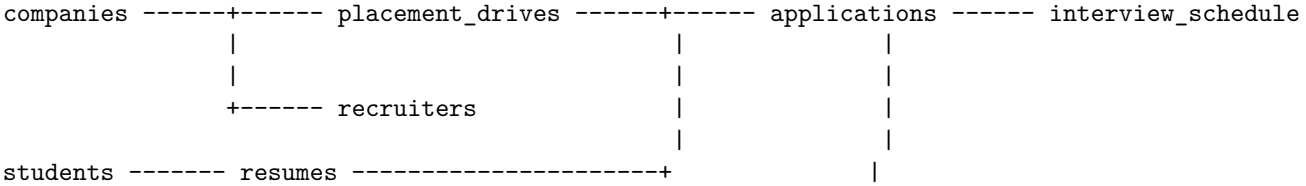
PlacePro persists all business data in a MySQL 8.0+ relational database accessed via JDBC. The schema comprises nine tables normalized to **Third Normal Form (3NF)** unless otherwise noted. All tables use the `InnoDB` storage engine to support foreign key constraints and transactional integrity (NFR-08, NFR-23).

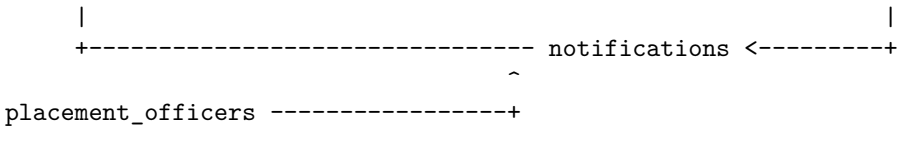
Design conventions:

Convention	Detail
Naming	Snake_case table and column names; singular entity roots (e.g., `students`, `companies`)
Primary keys	`INT AUTO_INCREMENT` surrogate keys (`*_id`)
Timestamps	`created_at` and `updated_at` on mutable entities
Soft delete	`is_active` flag where deactivation is preferred over hard delete
Passwords	Stored as BCrypt hashes only (NFR-04); never plaintext

Administrator accounts: The PRD defines a distinct Admin role (F3). In this schema, administrators are stored in the `placement_officers` table with `role = 'ADMIN'`, avoiding a redundant credentials table while preserving role separation at the application layer.

16.1 16.1 Entity-Relationship Overview





Relationship	Cardinality	Description
companies → placement_drives	1 : N	One company hosts many placement drives
companies → recruiters	1 : N	One company may have multiple recruiter accounts
students → resumes	1 : N	One student may upload multiple resume versions over time
students → applications	1 : N	One student may apply to many drives
placement_drives → applications	1 : N	One drive receives many applications
applications → interview_schedule	1 : N	One application may have multiple interview rounds
students / placement_officers / recruiters → notifications	1 : N	Each user type receives many notifications
resumes → applications	1 : N	One resume may be referenced by multiple applications (optional)

16.2 16.2 Table: `students`

Stores registered student accounts and academic eligibility attributes used by the Eligibility Checker (F10).

Column	Data Type	Constraints	Description
`student_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique student identifier
`roll_number`	VARCHAR(20)	NOT NULL, UNIQUE	Institutional roll number
`full_name`	VARCHAR(100)	NOT NULL	Student full name
`email`	VARCHAR(100)	NOT NULL, UNIQUE	Login email / contact email
`phone`	VARCHAR(15)	NOT NULL	Contact phone number
`password_hash`	VARCHAR(255)	NOT NULL	BCrypt-hashed password
`branch`	VARCHAR(50)	NOT NULL	Department / branch (e.g., CSE, ECE)
`cgpa`	DECIMAL(4,2)	NOT NULL	Current CGPA
`backlog_count`	INT	NOT NULL, DEFAULT 0	Number of active backlogs
`graduation_year`	INT	NOT NULL	Expected graduation year
`is_active`	TINYINT(1)	NOT NULL, DEFAULT 1	Account active flag
`created_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Registration timestamp
`updated_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last profile update

Primary Key: `student_id`

Foreign Keys: None

Relationships:

- Referenced by `resumes.student_id`

- Referenced by `applications.student_id`
- Referenced by `notifications.student_id`

Normalization (3NF): All attributes depend solely on `student_id`. Branch and CGPA are single-valued facts about the student, not repeating groups. No transitive dependencies exist (e.g., branch name does not determine CGPA). Meets **3NF**.

16.3 16.3 Table: `placement_officers`

Stores placement cell staff and administrator accounts (role-differentiated).

Column	Data Type	Constraints	Description
`officer_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique officer/admin identifier
`employee_id`	VARCHAR(20)	NOT NULL, UNIQUE	Institutional employee ID
`full_name`	VARCHAR(100)	NOT NULL	Officer or administrator name
`email`	VARCHAR(100)	NOT NULL, UNIQUE	Login email
`phone`	VARCHAR(15)	NULL	Contact phone
`password_hash`	VARCHAR(255)	NOT NULL	BCrypt-hashed password
`role`	ENUM('OFFICER','ADMIN')	NOT NULL, DEFAULT 'OFFICER'	Distinguishes placement officer from system administrator
`department`	VARCHAR(50)	NULL	Placement cell / department
`is_active`	TINYINT(1)	NOT NULL, DEFAULT 1	Account active flag
`created_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
`updated_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

Primary Key: `officer_id`

Foreign Keys: None

Relationships:

- Referenced by `interview_schedule.created_by_officer_id`
- Referenced by `notifications.officer_id`

Normalization (3NF): Each officer attribute is a direct fact about `officer_id`. The `role` attribute is a single-valued enum, not a repeating group. No transitive dependencies. Meets **3NF**.

16.4 16.4 Table: `recruiters`

Stores recruiter login accounts linked to a recruiting company.

Column	Data Type	Constraints	Description
`recruiter_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique recruiter identifier
`company_id`	INT	NOT NULL	Associated company
`full_name`	VARCHAR(100)	NOT NULL	Recruiter name
`email`	VARCHAR(100)	NOT NULL, UNIQUE	Login email
`phone`	VARCHAR(15)	NULL	Contact phone

Column	Data Type	Constraints	Description
password_hash	VARCHAR(255)	NOT NULL	BCrypt-hashed password
designation	VARCHAR(50)	NULL	Job title (e.g., Talent Acquisition Lead)
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	Account active flag
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

Primary Key: `recruiter_id`

Foreign Keys:

Column	References	On Delete
company_id	companies(company_id)	RESTRICT

Relationships:

- Belongs to `companies` (N : 1)
- Referenced by `interview_schedule.created_by_recruiter_id`
- Referenced by `notifications.recruiter_id`

Normalization (3NF): Recruiter personal details depend on `recruiter_id`; company affiliation is a foreign key reference, not a duplicated company attribute. Company name is not stored redundantly here. Meets **3NF**.

16.5 16.5 Table: `companies`

Stores recruiting company master records.

Column	Data Type	Constraints	Description
company_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique company identifier
company_name	VARCHAR(150)	NOT NULL, UNIQUE	Legal or brand company name
industry	VARCHAR(100)	NULL	Industry sector
contact_person	VARCHAR(100)	NULL	Primary HR / TPO contact at company
email	VARCHAR(100)	NULL	Company contact email
phone	VARCHAR(15)	NULL	Company contact phone
website	VARCHAR(200)	NULL	Company website URL
address	TEXT	NULL	Company address
is_active	TINYINT(1)	NOT NULL, DEFAULT 1	Active / deactivated flag
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

Primary Key: `company_id`

Foreign Keys: None

Relationships:

- Referenced by `recruiters.company_id`
- Referenced by `placement_drives.company_id`

Normalization (3NF): Company attributes are atomic and depend only on `company_id`. No multi-valued fields (e.g., multiple industries stored in one column). Meets **3NF**.

16.6 16.6 Table: `placement_drives`

Stores placement drive definitions with eligibility rules and lifecycle state.

Column	Data Type	Constraints	Description
`drive_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique drive identifier
`company_id`	INT	NOT NULL	Hosting company
`job_title`	VARCHAR(100)	NOT NULL	Position / role title
`job_description`	TEXT	NULL	Role description and requirements
`package_min`	DECIMAL(10,2)	NULL	Minimum CTC (in LPA or currency unit)
`package_max`	DECIMAL(10,2)	NULL	Maximum CTC
`min_cgpa`	DECIMAL(4,2)	NOT NULL	Minimum CGPA eligibility
`max_backlogs`	INT	NOT NULL, DEFAULT 0	Maximum allowed backlogs
`allowed_branches`	VARCHAR(500)	NOT NULL	Comma-separated list of eligible branches (e.g., `CSE,ECE,IT`)
`visit_date`	DATE	NULL	On-campus visit / drive date
`application_deadline`	DATETIME	NOT NULL	Last date/time to apply
`status`	ENUM('DRAFT','PUBLISHED','CLOSED','COMPLETED')	NOT NULL, DEFAULT 'DRAFT'	Drive lifecycle state
`created_by`	INT	NOT NULL	Officer who created the drive
`created_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Drive creation timestamp
`updated_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

Primary Key: `drive_id`

Foreign Keys:

Column	References	On Delete
`company_id`	`companies(company_id)`	RESTRICT
`created_by`	`placement_officers(officer_id)`	RESTRICT

Relationships:

- Belongs to `companies` (N : 1)
- Created by `placement_officers` (N : 1)
- Referenced by `applications.drive_id`

Normalization (3NF with note): Drive attributes depend on `drive_id`. Company details are not duplicated — only `company_id` is stored. The `allowed_branches` column is a deliberate **denormalized multi-value field** for v1.0 simplicity; a fully normalized alternative would be a junction table `drive_eligible_branches(drive_id, branch)`. This is documented as a known trade-off: the table otherwise meets **3NF** for all single-valued attributes.

16.7 16.7 Table: `resumes`

Stores resume file metadata; actual files reside on the local filesystem (NFR-26).

Column	Data Type	Constraints	Description
`resume_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique resume identifier
`student_id`	INT	NOT NULL	Owning student
`file_name`	VARCHAR(255)	NOT NULL	Original uploaded file name
`file_path`	VARCHAR(500)	NOT NULL	Server/local filesystem path
`file_type`	VARCHAR(10)	NOT NULL	MIME extension (PDF, DOC)
`file_size_kb`	INT	NOT NULL	File size in kilobytes
`is_current`	TINYINT(1)	NOT NULL, DEFAULT 1	Whether this is the student's active resume
`uploaded_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Upload timestamp

Primary Key: `resume_id`

Foreign Keys:

Column	References	On Delete
`student_id`	`students(student_id)`	CASCADE

Relationships:

- Belongs to `students` (N : 1)
- Referenced by `applications.resume_id`

Normalization (3NF): File metadata depends on `resume_id`. Student identity is referenced via FK, not duplicated. Binary file content is intentionally excluded from the database (stored on filesystem), avoiding BLOB redundancy. Meets **3NF**.

16.8 16.8 Table: `applications`

Stores student applications to placement drives with workflow status.

Column	Data Type	Constraints	Description
application_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique application identifier
student_id	INT	NOT NULL	Applying student
drive_id	INT	NOT NULL	Target placement drive
resume_id	INT	NULL	Resume submitted with application
status	ENUM('APPLIED','SHORTLISTED','INTERVIEW_SCHEDULED','SELECTED','REJECTED','ON_HOLD')	NOT NULL, DEFAULT 'APPLIED'	Current workflow status
applied_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Application submission time
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last status update time

Primary Key: `application_id`

Foreign Keys:

Column	References	On Delete
`student_id`	`students(student_id)`	RESTRICT
`drive_id`	`placement_drives(drive_id)`	RESTRICT
`resume_id`	`resumes(resume_id)`	SET NULL

Unique Constraints:

Constraint	Columns	Purpose
`uk_student_drive`	(`student_id`, `drive_id`)	Prevents duplicate applications (FR-24)

Relationships:

- Belongs to `students` (N : 1)
- Belongs to `placement_drives` (N : 1)
- Optionally references `resumes` (N : 1)
- Referenced by `interview_schedule.application_id`

Normalization (3NF): Application status is a fact about the student-drive pair, identified by `application_id`. Student and drive details are not duplicated. Status history (FR-27) may be extended via a separate `application_status_history` table in a future iteration; current design meets **3NF** for the core application record.

16.9 16.9 Table: `interview_schedule`

Stores scheduled interview rounds and outcomes for shortlisted applications.

Column	Data Type	Constraints	Description
interview_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique interview record identifier
application_id	INT	NOT NULL	Associated application
round_number	INT	NOT NULL, DEFAULT 1	Interview round sequence (1, 2, 3…)
round_type	VARCHAR(50)	NULL	Round label (e.g., Aptitude, Technical, HR)
scheduled_date	DATE	NOT NULL	Interview date
scheduled_time	TIME	NOT NULL	Interview start time
venue	VARCHAR(150)	NULL	Interview room / location
outcome	ENUM('PENDING','SELECTED','REJECTED','ON_HOLD')	NOT NULL, DEFAULT 'PENDING'	Round outcome
notes	TEXT	NULL	Officer or recruiter notes
created_by_officer_id	INT	NULL	Officer who created the schedule
created_by_recruiter_id	INT	NULL	Recruiter who created the schedule
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last update timestamp

Primary Key: `interview_id`

Foreign Keys:

Column	References	On Delete
application_id	applications(application_id)	CASCADE
created_by_officer_id	placement_officers(officer_id)	SET NULL
created_by_recruiter_id	recruiters(recruiter_id)	SET NULL

Unique Constraints:

Constraint	Columns	Purpose
uk_application_round	(application_id, round_number)	One record per round per application

Relationships:

- Belongs to `applications` (N : 1)
- Optionally created by `placement_officers` (N : 1)
- Optionally created by `recruiters` (N : 1)

Normalization (3NF): Interview details depend on `interview_id`. Application, student, and drive data are accessed through `application_id` FK, not duplicated. Creator references use FKs rather than storing officer/recruiter names inline. Meets **3NF**.

16.10 16.10 Table: `notifications`

Stores in-application notifications for students, officers, and recruiters (F14).

Column	Data Type	Constraints	Description
`notification_id`	INT	PRIMARY KEY, AUTO_INCREMENT	Unique notification identifier
`student_id`	INT	NULL	Recipient student (if applicable)
`officer_id`	INT	NULL	Recipient officer/admin (if applicable)
`recruiter_id`	INT	NULL	Recipient recruiter (if applicable)
`title`	VARCHAR(150)	NOT NULL	Notification headline
`message`	TEXT	NOT NULL	Notification body
`notification_type`	ENUM('DRIVE_PUBLISHED','STATUS_CHANGE','INTERVIEW_SCHEDULED','GENERAL')	NOT NULL	Event category
`reference_id`	INT	NULL	Related entity ID (drive, application, or interview)
`is_read`	TINYINT(1)	NOT NULL, DEFAULT 0	Read / unread flag
`created_at`	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Notification creation time

Primary Key: `notification_id`

Foreign Keys:

Column	References	On Delete
`student_id`	`students(student_id)`	CASCADE
`officer_id`	`placement_officers(officer_id)`	CASCADE
`recruiter_id`	`recruiters(recruiter_id)`	CASCADE

Check Constraint (application layer): Exactly one of `student_id`, `officer_id`, or `recruiter_id` must be non-null per notification record.

Relationships:

- Optionally belongs to `students` (N : 1)
- Optionally belongs to `placement_officers` (N : 1)
- Optionally belongs to `recruiters` (N : 1)

Normalization (3NF): Notification content depends on `notification_id`. Recipient identity uses nullable FKs to the three user tables rather than a generic polymorphic key without referential integrity. The `reference_id` is a loose coupling to related entities (enforced at application layer). Meets **3NF** for stored attributes.

16.11 16.11 Schema Summary

#	Table	Primary Key	Foreign Keys	Row Estimate (Season)
1	`students`	`student_id`	—	~500
2	`placement_officers`	`officer_id`	—	~5–10
3	`recruiters`	`recruiter_id`	`company_id`	~30–50
4	`companies`	`company_id`	—	~30–50
5	`placement_drives`	`drive_id`	`company_id`, `created_by`	~50
6	`resumes`	`resume_id`	`student_id`	~500–1,000
7	`applications`	`application_id`	`student_id`, `drive_id`, `resume_id`	~2,000–5,000
8	`interview_schedule`	`interview_id`	`application_id`, `created_by_officer_id`, `created_by_recruiter_id`	~1,000–3,000
9	`notifications`	`notification_id`	`student_id`, `officer_id`, `recruiter_id`	~5,000–10,000

16.12 16.12 Normalization Summary

Normal Form	Status	Explanation
1NF	Met	All columns contain atomic values. No repeating groups within a row (except `allowed_branches`, noted below).
2NF	Met	All non-key attributes depend on the full primary key. No composite PKs with partial dependencies.
3NF	Met	No transitive dependencies. Entity details are stored in their owning table; related entities referenced by FK only.
Known trade-off	`placement_drives.allowed_branches`	Comma-separated branch list is a pragmatic v1.0 denormalization. A `drive_eligible_branches` junction table would achieve full multi-value normalization if required in a future release.

Total tables: 9 — aligned with functional requirements for JDBC persistence (NFR-22), referential integrity (NFR-23), and third normal form (NFR-19).

---

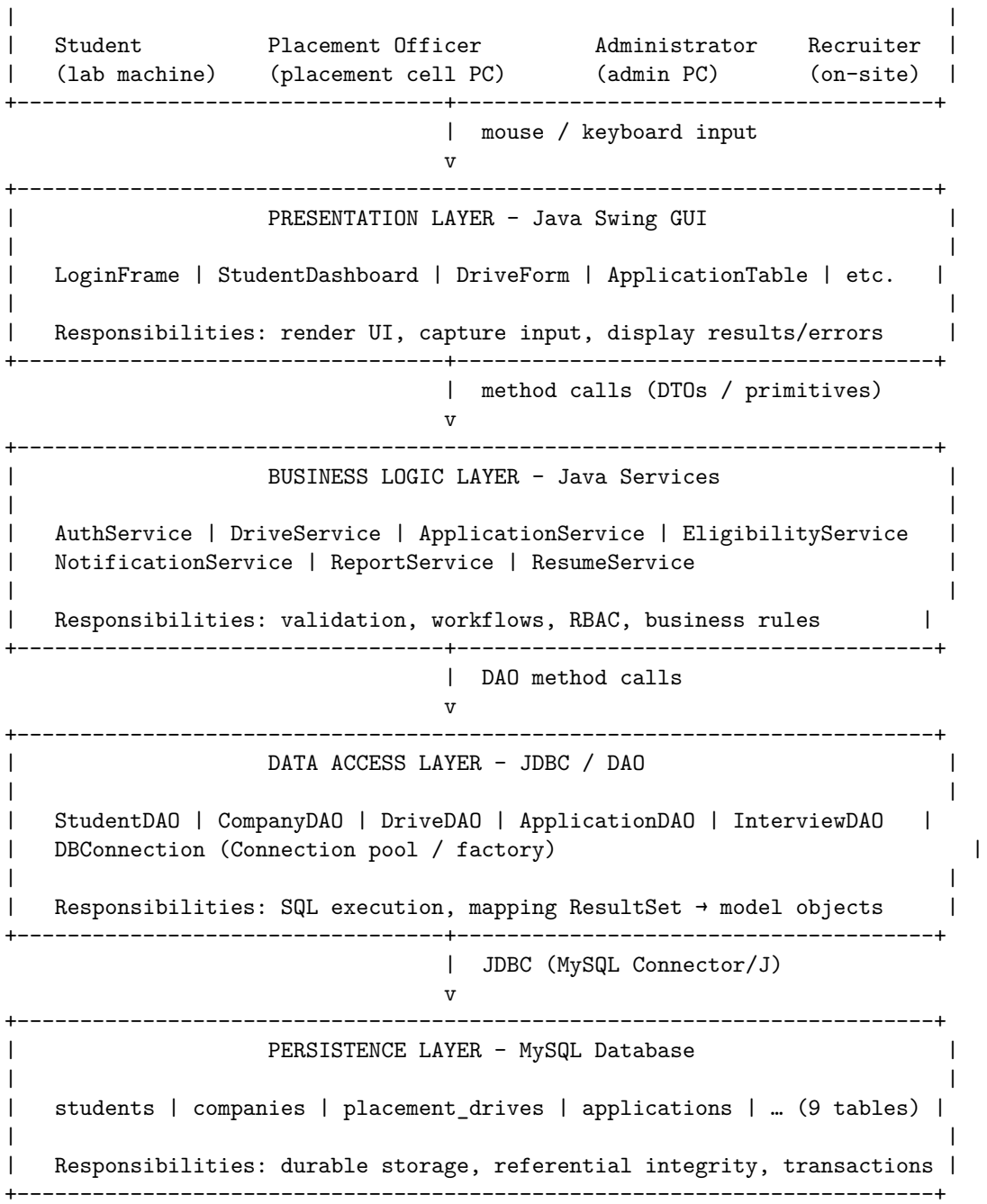
17 17. System Architecture

PlacePro follows a **layered desktop architecture** in which each tier has a single, well-defined responsibility. User interactions originate at the Java Swing presentation layer, pass through business logic for validation and workflow enforcement, descend to the JDBC data access layer for database operations, and terminate in the MySQL database where all persistent state resides. No web server, middleware broker, or cloud service participates in the request path.

This design satisfies the maintainability requirement (NFR-15) by enforcing separation of concerns: the UI never executes SQL directly, the business layer never renders Swing components, and the JDBC layer contains no placement domain rules.

17.1 17.1 Architecture Overview

```
+-------------------------------------------------------------------------+
|                                USER LAYER                               |
```



17.2 17.2 Layer Responsibilities

Layer	Technology	Responsibility
User	Human operators	Initiates actions (login, apply, review, schedule) via desktop workstation
Java Swing GUI	Java Swing (`javax.swing`)	Renders screens, captures form input, displays tables/reports, shows validation errors

Layer	Technology	Responsibility
Business Logic	Java service classes	Enforces domain rules, orchestrates workflows, applies RBAC, coordinates multi-step operations
JDBC Layer	JDBC + DAO classes	Executes parameterized SQL, manages connections, maps rows to Java model objects
MySQL Database	MySQL 8.0+	Stores all persistent entities with FK constraints and transactional guarantees

---

17.3 17.3 Layer 1 — User

The user layer comprises the four stakeholder roles defined in Section 10 who interact with PlacePro through desktop workstations on the college local network.

Role	Typical Actions
Student	Register, log in, upload resume, check eligibility, apply to drives, view notifications
Placement Officer	Manage companies and drives, review applications, schedule interviews, generate reports
Administrator	Manage user accounts, configure system settings, view analytics
Recruiter	Log in, view shortlists, schedule interviews, record outcomes

Users interact exclusively through the Swing GUI. They have no direct access to the database, JDBC layer, or business logic source code. All input is mediated through form controls (`JTextField`, `JComboBox`, `JButton`, `JTable`) and all output is rendered as Swing components or dialog messages.

17.4 17.4 Layer 2 — Java Swing GUI (Presentation Layer)

The presentation layer is responsible for everything the user sees and touches. It contains no SQL statements and no business rule evaluation beyond basic field-format checks (e.g., non-empty required fields).

Responsibilities:

- Render role-specific screens: login forms, dashboards, data entry forms, searchable tables, report viewers
- Capture user input and package it into Java objects (DTOs or model instances) for the business layer
- Invoke business logic service methods in response to button clicks and menu selections
- Display results returned by the business layer: populated tables, success confirmations, error dialogs
- Enforce UI-level input validation before forwarding requests (NFR-30)
- Route navigation based on authenticated role (student portal vs. officer console vs. admin panel)
- Handle global uncaught exceptions with a user-friendly error dialog (NFR-31)

Key packages (illustrative):

Package	Contents
`com.placepro.ui`	Main application frame, navigation
`com.placepro.ui.login`	Login and registration screens
`com.placepro.ui.student`	Student dashboard, apply, tracking screens
`com.placepro.ui.officer`	Drive management, application review screens
`com.placepro.ui.admin`	User management, analytics screens
`com.placepro.ui.common`	Shared dialogs, table renderers, validators

Boundary rule: Swing classes call service interfaces only. They never import or instantiate DAO classes directly.

17.5 17.5 Layer 3 — Business Logic (Service Layer)

The business logic layer is the core of PlacePro's placement domain intelligence. It sits between the UI and the database, enforcing every rule defined in the functional requirements (Section 13).

Responsibilities:

- **Authentication and authorization** — Validate credentials, establish sessions, enforce RBAC per role (FR-04 – FR-11, NFR-05)
- **Eligibility evaluation** — Compare student academic profile against drive rules before allowing application (F10, FR-21 – FR-22)
- **Application workflow** — Manage status transitions (Applied → Shortlisted → Interview Scheduled → Selected/Rejected), prevent duplicate applications (FR-23 – FR-27)
- **Drive lifecycle** — Enforce valid state transitions (Draft → Published → Closed → Completed) (FR-14 – FR-16)
- **Interview coordination** — Create interview records, associate rounds with applications (FR-28 – FR-29)
- **Notification dispatch** — Generate in-app notifications on status changes and drive events (FR-30 – FR-31)
- **Report and analytics generation** — Aggregate data for reports and dashboard KPIs (FR-32 – FR-35)
- **Resume management** — Validate file type/size, coordinate filesystem storage with database metadata (FR-19 – FR-20)
- **Transaction orchestration** — Coordinate multi-step DAO operations within a single JDBC transaction (NFR-08)

Key packages (illustrative):

Package	Contents
`com.placepro.service`	Service interfaces and implementations
`com.placepro.service.auth`	AuthService, SessionManager
`com.placepro.service.drive`	DriveService, EligibilityService
`com.placepro.service.application`	ApplicationService, InterviewService
`com.placepro.service.notification`	NotificationService
`com.placepro.service.report`	ReportService, AnalyticsService
`com.placepro.model`	Plain Java entity classes (Student, Drive, Application, etc.)

Boundary rule: Service classes call DAO interfaces. They never create JDBC `Connection` objects or write SQL strings.

17.6 17.6 Layer 4 — JDBC Layer (Data Access Layer)

The JDBC layer is the sole bridge between Java application logic and the MySQL database. Every database read and write passes through this layer.

Responsibilities:

- **Connection management** — Obtain and release JDBC `Connection` objects via a `DBConnection` factory reading credentials from configuration (NFR-07, NFR-11)

- **CRUD operations** — Implement Create, Read, Update, Delete for each entity through dedicated DAO classes
- **Parameterized queries** — Execute all SQL via `PreparedStatement` to prevent injection (NFR-24)
- **Result mapping** — Convert `ResultSet` rows into Java model objects returned to the service layer
- **Transaction support** — Accept `Connection` from service layer for transactional commit/rollback on multi-step writes (NFR-08)
- **Exception translation** — Catch `SQLException`, log details (NFR-27), and throw application-specific data access exceptions with user-safe messages (NFR-29)

Key packages (illustrative):

Package	Contents
`com.placepro.dao`	DAO interfaces and JDBC implementations
`com.placepro.dao.impl`	StudentDAOImpl, DriveDAOImpl, ApplicationDAOImpl, etc.
`com.placepro.util`	DBConnection, DateUtil, PasswordUtil (BCrypt)

Boundary rule: DAO classes contain SQL and JDBC code only. They contain no Swing imports and no business rule logic (e.g., no eligibility checks).

17.7 17.7 Layer 5 — MySQL Database (Persistence Layer)

The persistence layer is the authoritative source of truth for all PlacePro data, as defined in Section 16.

Responsibilities:

- **Durable storage** — Persist all nine entity tables with referential integrity enforced by foreign keys (NFR-23)
- **Concurrency** — Handle simultaneous read/write access from multiple desktop clients connected to the same MySQL server on the LAN
- **Data integrity** — Enforce unique constraints (e.g., one application per student per drive), NOT NULL rules, and ENUM validity
- **Transactional atomicity** — Support InnoDB transactions so multi-table writes (application + notification) succeed or fail as a unit
- **Backup and recovery** — Serve as the target for `mysqldump` backup procedures (NFR-25)

Boundary rule: The database stores data only. Business logic (eligibility rules, status workflows) lives in the service layer, not in stored procedures, for v1.0.

17.8 17.8 Data Flow Between Layers

Data flows **downward** on requests (user action → database write/read) and **upward** on responses (database result → UI display). The following table describes the flow at each boundary.

Boundary	Downstream (Request)	Upstream (Response)
User → Swing GUI	Mouse clicks, keyboard input, file selection	Rendered screens, tables, dialogs, error messages
Swing GUI → Business Logic	Java method calls with DTOs / primitives (e.g., `applicationService.apply(studentId, driveId)`)	Result objects, success/failure enums, lists of entities

Boundary	Downstream (Request)	Upstream (Response)
Business Logic → JDBC Layer	DAO method calls (e.g., `applicationDAO.insert(application)`)	Populated model objects, row counts, generated keys
JDBC Layer → MySQL	Parameterized SQL via `PreparedStatement` over JDBC connection	`ResultSet` rows, affected row counts, SQL exceptions

Request flow (write operation):

```
User clicks "Apply"
    → Swing: ApplyDrivePanel captures studentId, driveId
        → Service: ApplicationService.apply(studentId, driveId)
            → validates eligibility (EligibilityService)
            → checks duplicate (ApplicationDAO.exists)
            → DAO: ApplicationDAO.insert(application)
                → JDBC: INSERT INTO applications (…) VALUES (…)
                    → MySQL: row persisted
            → DAO: NotificationDAO.insert(notification)
                → JDBC: INSERT INTO notifications (…)
                    → MySQL: row persisted
        → Service: returns ApplicationResult.SUCCESS
    → Swing: shows confirmation dialog, refreshes Application Tracking table
```

Response flow (read operation):

```
User opens Student Dashboard
    → Swing: StudentDashboardPanel.onLoad()
        → Service: DashboardService.getSummary(studentId)
            → DAO: ApplicationDAO.countByStatus(studentId)
                → JDBC: SELECT status, COUNT(*) … GROUP BY status
                    → MySQL: returns aggregated rows
            → DAO: DriveDAO.findPublished()
                → JDBC: SELECT * FROM placement_drives WHERE status = 'PUBLISHED'
                    → MySQL: returns drive rows
        → Service: returns DashboardSummary object
    → Swing: populates JLabel counters and JTable drive listing
```

17.9 17.9 End-to-End Example — Student Applies to a Drive

The following sequence traces a complete primary use flow (Section 12.1, Steps 5–6) through all architectural layers.

Step	Layer	Action
1	**User**	Student selects a drive and clicks **Apply**
2	**Swing GUI**	`ApplyDrivePanel` reads `studentId` from session and `driveId` from selected table row; calls `applicationService.submitApplication(studentId, driveId)`
3	**Business Logic**	`ApplicationService` calls `eligibilityService.check(studentId, driveId)` — if ineligible, returns error without touching database
4	**Business Logic**	`ApplicationService` calls `applicationDAO.exists(studentId, driveId)` — if duplicate, returns error
5	**JDBC Layer**	`ApplicationDAO.exists()` executes `SELECT COUNT(*) FROM applications WHERE student_id = ? AND drive_id = ?`

Step	Layer	Action
6	**MySQL**	Returns count = 0
7	**Business Logic**	`ApplicationService` builds `Application` object with status `APPLIED`, calls `applicationDAO.insert(application)` within a transaction
8	**JDBC Layer**	`ApplicationDAO.insert()` executes parameterized `INSERT INTO applications (…)` and returns generated `application_id`
9	**MySQL**	Persists new application row; FK constraints validate `student_id` and `drive_id`
10	**Business Logic**	`NotificationService.createStatusNotification(studentId, …)` inserts notification in same transaction; commits
11	**Swing GUI**	Receives `ApplicationResult.SUCCESS`; displays confirmation dialog; navigates to Application Tracking view
12	**User**	Student sees application listed with status **Applied**

17.10 17.10 Cross-Cutting Concerns

The following concerns span multiple layers but are anchored to a primary layer of ownership.

Concern	Primary Layer	How It Is Applied
Authentication	Business Logic	`AuthService` validates credentials via `StudentDAO` / `OfficerDAO`; session token held in memory
Role-based access	Business Logic + Swing	Service methods check role before executing; Swing shows/hides menus per role
Input validation	Swing + Business Logic	UI validates format; service validates business rules (eligibility, duplicates)
Error handling	All layers	DAO catches SQL exceptions; service translates to domain errors; Swing shows user messages
Logging	JDBC + Business Logic	`Logger` writes auth events, SQL errors, and unhandled exceptions to log file (NFR-27)
Configuration	JDBC Layer	`DBConnection` reads host, port, database, username, password from `config.properties`
Resume files	Business Logic + filesystem	`ResumeService` writes file to disk; `ResumeDAO` stores metadata path in MySQL

17.11 17.11 Architecture Constraints

Constraint	Detail
No layer skipping	Swing must not call DAO directly; all database access goes through the service layer
No upward dependencies	DAO does not import Swing or service classes; dependencies flow downward only
Single database	All DAO classes connect to one MySQL instance; no distributed data stores in v1.0
Desktop process	Entire application runs as one JVM process per workstation; no client-server split across machines for the Java layers
Shared MySQL server	Multiple desktop clients may connect to the same MySQL server on the campus LAN

17.12 17.12 Alignment with Requirements

Requirement	Architectural Support
NFR-15 (Layered architecture)	Five distinct layers with explicit package boundaries
NFR-08 (Transactions)	Service layer coordinates transactional JDBC operations
NFR-24 (SQL injection prevention)	JDBC layer uses PreparedStatements exclusively
NFR-05 (RBAC)	Business logic enforces role checks; Swing renders role-specific views
NFR-22 (MySQL persistence)	All business data flows through JDBC to MySQL
FR-01 – FR-39	Each functional requirement maps to a service method backed by one or more DAO operations

---

18 18. Risk Assessment

This section identifies the principal risks associated with developing, deploying, and operating PlacePro as a Java Swing desktop application backed by MySQL. Each risk is rated by **Likelihood** and **Impact**, from which an overall **Severity** is derived. Mitigation strategies are specified as concrete, actionable measures aligned with the non-functional requirements in Section 14.

Rating scales:

Dimension	Levels
Likelihood	High — probable during normal operation; Medium — may occur under stress or misuse; Low — unlikely but possible
Impact	High — data loss, security breach, or placement season disruption; Medium — degraded experience or recoverable errors; Low — minor inconvenience
Severity	Critical — immediate threat to data integrity or project viability; High — significant disruption requiring urgent action; Medium — manageable with planned mitigation; Low — acceptable residual risk

---

18.1 18.1 Risk Register

Risk ID	Risk	Likelihood	Impact	Severity	Mitigation
R1	**Database server failure** — MySQL instance becomes unavailable due to hardware fault, misconfiguration, or service crash during active placement season, rendering the desktop application unable to read or write data.	Medium	High	**High**	Configure automated MySQL service restart; maintain daily `mysqldump` backups (NFR-25); implement connection retry logic with user-friendly error dialogs (NFR-11); document recovery procedure in repository README.

Risk ID	Risk	Likelihood	Impact	Severity	Mitigation
R2	**Data loss without backup** — Placement records, applications, and interview outcomes are lost due to absent or untested backup procedures, disk failure, or accidental `DROP TABLE` during development.	Medium	High	**Critical**	Enforce pre-season full backup via `mysqldump`; store backups on a separate drive or machine; test restore procedure before placement season; restrict DROP privileges on production database accounts.
R3	**SQL injection vulnerability** — User input concatenated into SQL strings in DAO classes could allow malicious input to alter or exfiltrate database records.	Low	High	**High**	Mandate parameterized `PreparedStatement` for all queries (NFR-24); conduct code review focused on DAO layer; prohibit string-concatenated SQL in coding standards.
R4	**Exposed database credentials** — MySQL username and password hardcoded in source code or committed to the GitHub repository, exposing the database to unauthorized access.	Medium	High	**High**	Store credentials in `config.properties` excluded via `.gitignore` (NFR-07); use environment-specific config files; rotate credentials if exposure is detected; never commit secrets to GitHub.
R5	**Weak password storage** — User passwords stored in plaintext or with weak hashing, enabling credential theft if the database is compromised.	Low	High	**High**	Hash all passwords with BCrypt (work factor >= 10) before storage (NFR-04, FR-03); never log or display plaintext passwords; validate hashing in unit tests.
R6	**Concurrent access conflicts** — Multiple desktop clients (students and officers) writing to the same application or drive record simultaneously, causing lost updates or inconsistent status values.	Medium	Medium	**Medium**	Use InnoDB row-level locking and transactional JDBC operations (NFR-08); implement optimistic concurrency via `updated_at` timestamp checks; serialize critical officer workflows (shortlisting) at the service layer.
R7	**Session persistence on shared machines** — A student or officer forgets to log out on a shared lab workstation, allowing the next user to access their placement session and data.	High	Medium	**High**	Implement explicit Logout (F19, FR-38); display session timeout warning after inactivity (e.g., 15 minutes); clear session state on logout and application close; prompt "Are you still there?" dialog on idle timeout.
R8	**Swing UI thread blocking** — Long-running database queries or report generation executed on the Event Dispatch Thread (EDT) freeze the desktop UI, causing the application to appear hung during peak usage.	Medium	Medium	**Medium**	Offload long operations (report generation, bulk search) to `SwingWorker` background threads; show progress indicators during async tasks; enforce EDT rule in code review checklist.
R9	**Resume file storage failure** — Uploaded resume files become inaccessible due to incorrect file paths, missing upload directory, or disk space exhaustion on the local filesystem.	Medium	Medium	**Medium**	Validate upload directory exists at startup; enforce file size limits (FR-19); store absolute paths in `resumes` table; include resume directory in backup procedure alongside database (NFR-26); monitor disk space on deployment machine.
R10	**Role-based access bypass** — Insufficient enforcement of RBAC at the service layer allows a student to invoke officer-only operations (e.g., status updates, drive creation) through direct service calls or UI manipulation.	Low	High	**High**	Enforce role checks in every service method before executing privileged operations (NFR-05); never rely on UI menu hiding alone; add integration tests verifying unauthorized role access is rejected.

Risk ID	Risk	Likelihood	Impact	Severity	Mitigation
R11	**JDBC connection leaks** — Unclosed `Connection`, `Statement`, or `ResultSet` objects exhaust the MySQL connection pool, eventually preventing all clients from connecting.	Medium	High	**High**	Use try-with-resources for all JDBC objects; centralize connection lifecycle in `DBConnection` factory; log active connection count; load-test with 10+ simultaneous clients during development.
R12	**Eligibility rule inconsistency** — The comma-separated `allowed_branches` field in `placement_drives` is entered inconsistently (e.g., `CSE` vs. `cse` vs. `Computer Science`), causing eligible students to be incorrectly rejected.	Medium	Medium	**Medium**	Normalize branch codes to a standard enum list in the application layer; validate branch values at drive creation; provide a dropdown for branch selection rather than free-text entry; document branch code conventions in README.
R13	**Single point of failure — one MySQL server** — All desktop clients depend on a single MySQL instance; server downtime halts all placement operations across the institution.	Medium	High	**High**	Schedule MySQL maintenance outside placement season; configure automatic service recovery; maintain tested backup for rapid restore; document maximum acceptable downtime; consider read-replica for reporting in future versions.
R14	**Insufficient input validation** — Invalid or malformed user input (negative CGPA, future dates, empty required fields) is persisted to the database, corrupting eligibility checks and reports.	Medium	Medium	**Medium**	Implement UI-level field validation before submission (NFR-30); enforce business-rule validation in service layer; apply MySQL column constraints (NOT NULL, CHECK where supported); display field-level error messages without data loss.

18.2 18.2 Risk Summary by Severity

Severity	Risk IDs	Count
Critical	R2	1
High	R1, R3, R4, R5, R7, R10, R11, R13	8
Medium	R6, R8, R9, R12, R14	5
Low	—	0

18.3 18.3 Risk Priority Matrix

The matrix below maps risks by Likelihood and Impact to support prioritization during development sprints and pre-deployment review.

	Low Impact	Medium Impact	High Impact
High Likelihood	—	R7	—
Medium Likelihood	—	R6, R8, R9, R12, R14	R1, R2, R4, R11, R13
Low Likelihood	—	—	R3, R5, R10

18.4 18.4 Mitigation Priority for Development

Based on the risk register, the following mitigations should be implemented **before** placement season deployment, listed in priority order:

Priority	Risk ID	Mitigation Action	Related NFR/FR
1	R2	Implement and test `mysqldump` backup/restore procedure	NFR-25
2	R4, R5	Externalize credentials; implement BCrypt password hashing	NFR-04, NFR-07
3	R3, R11	Code review: PreparedStatements only; try-with-resources for JDBC	NFR-24, NFR-08
4	R10	Add RBAC checks to all service methods; write access-control tests	NFR-05
5	R7	Implement logout, session timeout, and idle warning dialog	FR-38, FR-39
6	R1, R13	Connection retry logic; MySQL service auto-restart configuration	NFR-11
7	R8	Move report generation and bulk queries to `SwingWorker` threads	NFR-02
8	R9	Validate upload directory at startup; enforce file size limits	NFR-26, FR-19
9	R12, R14	Branch code normalization; comprehensive input validation	NFR-30, FR-03
10	R6	Transactional writes for multi-step operations; `updated_at` concurrency checks	NFR-08

18.5 18.5 Residual Risk Acceptance

The following risks are accepted as low residual impact for v1.0 given the college project scope and deployment scale (<= 500 students, local network):

Risk	Acceptance Rationale
R13 — Single MySQL server	Acceptable at institutional scale; replication is out of scope (Section 7.2)
R6 — Concurrent conflicts	Low probability of simultaneous edits to the same record; transactional integrity limits damage
R12 — Branch code inconsistency	Mitigated by dropdown normalization; residual risk from legacy data entry errors

All other High and Critical risks must have documented mitigations in place before the application is used during a live placement season.

---

19 19. Future Enhancements

The following capabilities are **out of scope for PlacePro v1.0** (see Section 7.2) but represent a logical roadmap for evolving the platform beyond the initial Java Swing desktop release. Each enhancement builds on the v1.0 data model and workflows documented in this PRD.

#	Enhancement	Value Proposition
1	**Email Notifications**	Automatically deliver drive announcements, application status changes, and interview schedules to students and recruiters via email, reducing dependence on in-app checks and ensuring timely communication during high-pressure placement windows.
2	**SMS Notifications**	Send concise SMS alerts for critical events such as shortlist confirmations and interview reminders, reaching students who may not actively monitor the desktop application or email inbox.
3	**AI Resume Analysis**	Parse uploaded resumes using natural language processing to extract skills, projects, and experience, giving placement officers structured candidate summaries and reducing manual resume review effort.
4	**Online Coding Tests**	Integrate a built-in coding assessment module so companies can conduct aptitude and programming rounds remotely before on-campus visits, streamlining shortlisting and saving interview slot time.
5	**Video Interviews**	Embed or integrate video conferencing for virtual HR and technical interview rounds, enabling companies to engage candidates who cannot attend on-campus drives without relying on external scheduling tools.
6	**Cloud Deployment**	Migrate the application and database to a cloud-hosted environment (e.g., institutional VPS or managed MySQL), enabling secure remote access for officers and students without local network dependency or per-machine installation.
7	**Mobile Application**	Provide a companion Android or iOS app for students to browse drives, apply, and receive notifications on personal devices, significantly improving accessibility compared to lab-bound desktop access alone.
8	**Resume Score**	Assign a quantitative compatibility score by comparing parsed resume content against drive job requirements, helping officers and recruiters prioritize candidates with the strongest skill-to-role alignment.
9	**Placement Analytics**	Extend the v1.0 analytics dashboard with trend analysis, year-over-year comparisons, department benchmarking, and predictive placement rate modeling to support strategic decisions by institutional leadership.
10	**Company Portal**	Offer recruiters a dedicated self-service web portal to register drives, review applications, schedule interviews, and submit outcomes directly, reducing placement cell coordination overhead and email-based data exchange.
11	**Student Skill Assessment**	Allow students to complete standardized skill assessments (technical quizzes, aptitude tests) within PlacePro, storing verified scores on their profile to strengthen applications and support objective eligibility criteria.

19.1 19.1 Enhancement Priority Tiers

Tier	Enhancements	Rationale
Tier 1 — Near-term	Email Notifications, Company Portal, Placement Analytics	Low architectural disruption; extends existing notification and reporting foundations from v1.0
Tier 2 — Mid-term	SMS Notifications, Resume Score, Student Skill Assessment, Mobile Application	Requires external integrations or additional client platforms; builds on core data model
Tier 3 — Long-term	AI Resume Analysis, Online Coding Tests, Video Interviews, Cloud Deployment	Significant new subsystems or infrastructure changes; highest development and operational complexity

19.2 19.2 Dependency on v1.0 Foundation

All future enhancements depend on the v1.0 layered architecture (Section 17), normalized database schema (Section 16), and role-based access model remaining stable. The service and DAO layers should be designed

in v1.0 to accommodate extension — for example, `NotificationService` should expose an interface that an email or SMS adapter can implement without modifying core business logic.

19.3 Appendix A — Document Style Guide

Internal reference documenting the formatting conventions used to produce this PRD.

Derived from analysis of a sample college-project PRD. Use these patterns for PlacePro; do not copy sample content.

Aspect	Pattern to follow
Document structure	Cover metadata block (version, date, owner, status) → numbered sections in fixed order: Overview → Objectives → Target Users → Problem Statement → Proposed Solution → Features → Use Flow → Functional Requirements → Non-Functional Requirements → Tech Stack → Risk Assessment. Append-only across steps.
Writing tone	Professional, formal, third-person. Declarative and solution-oriented. Suitable for academic submission and demo/exhibition context. Lead sections with a labeled hook (e.g., *One-liner:*, *Core Problem:*).
Section hierarchy	Top-level sections numbered `1`, `2`, `3`… Subsections use decimals (`6.1`, `7.1`, `9.1`). Features live under a dedicated Features section; flows under Use Flow; NFRs grouped by category.
Formatting style	Markdown tables for metadata, personas, requirements, tech stack, and risks. Bullet lists for feature details and NFR items. Numbered lists for objectives and flow steps. ASCII/text diagram for architecture where helpful. Horizontal rules between major blocks.
Requirement numbering	Functional requirements: `FR-01`, `FR-02`, … (zero-padded, continuous across document). Table columns: ID
Feature numbering	Features: `F1`, `F2`, … under subsection headings formatted as `F{n} - Descriptive Name`. Each feature followed by bullet sub-points (capabilities, not copied wording).
Functional requirement style	Phrase as *"System shall …"* or *"The [component] shall …"*. Tie each FR to a named feature. Assign priority: **High**, **Medium**, or **Low**.
Non-functional requirement style	Group under named categories (e.g., Performance, Security, Reliability, Usability, Maintainability, Portability). Use bullets with measurable targets where possible (latency, load time, concurrency). Use *shall* / *must* for binding statements.
Risk assessment style	Risks: `R1`, `R2`, … with short title + em-dash + description. Table: #
Tech stack presentation	Table: Layer
User flow style	Section titled **Use Flow**. Subsections: *Primary Flow — [Scenario Name]* and *Secondary Flow — [Scenario Name]*. Numbered steps, action-oriented (verb first). Steps reflect realistic demo paths; include routes/roles where relevant.

— End of PlacePro Product Requirements Document v1.0 —